# 3D Interior Layout Generation from CAD-Based Floor Plan Files

*MAJOR PROJECT REPORT submitted in partial fulfillment of the requirements for the Award of the Degree of*

**BACHELOR OF TECHNOLOGY**

**In**

**INFORMATION TECHNOLOGY**

**By**


**HARIKA DAMERA - 228W1A1280**
**YASHMITHA KODE – 228W1A1298**

*Under the Guidance of*

**Dr. Y. Sandeep**

**Sr. Assistant Professor**

**DEPARTMENT OF INFORMATION TECHNOLOGYV R SIDDHARTHA ENGINEERING COLLEGE**

**(AUTONOMOUS - AFFILIATED TO JNTU-K, KAKINADA)**

**Approved by AICTE &Accredited by NBA**

**KANURU, VIJAYAWADA-520007ACADEMIC YEAR**


**(2025-26)**

# V.R. SIDDHARTHA ENGINEERING COLLEGE

(Affiliated to JNTUK: Kakinada, Approved by AICTE, Autonomous)(An ISO certified and NBA accredited institution)
Kanuru, Vijayawada – 520007

## CERTIFICATE

This is to certify that this project report titled **"3D Interior Layout Generation from CAD-Based Floor Plan Files"** is a Bonafide record of work done by **Damera Harika (228W1A1285), Kode Yashmitha (228W1A1298)** under my guidance and supervision is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, **V.R. Siddhartha EngineeringCollege** (Autonomous under JNTUK) during the year 2025-26.

| | |
|---|---|
| **(Dr. Y. Sandeep)** | **(Dr. M. Suneetha)** |
| Sr.Assistant Professor | Dean R&D, Professor &Head |
| Dept. of Information Technology | Dept. of Information Technology |

**EXTERNAL EXAMINER SIGNATURE**

*Date of examination:*

# DEPARTMENT OF INFORMATION TECHNOLOGY

# V.R. SIDDHARTHA ENGINEERING COLLEGE

## PROJECT SUMMARY

| S. No | Item | Description |
|---|---|---|
| 1 | **Project Title** | **3D Interior Layout Generation from CAD-Based Floor Plan Files** |
| 2 | **Student Names & Numbers** | **DAMERA HARIKA(228W1A1285)**<br>**KODE YASHMITHA(228W1A1298)** |
| 3 | **Name of The Guide** | **Dr. Y. Sandeep, Sr.Assistant Professor** |
| 4 | **Research Group** | **Vision-Based 3D Reconstruction and Geometric Processing** |
| 5 | **Application Area** | **Architecture, Interior Design, and 3D Visualization Systems** |
| 6 | **Aim of the Project** | To develop a system that converts CAD-based floor plans into structured data and generates interactive 3D interior layouts automatically. |
| 7 | **Project Outcomes** | Extract walls, corners, doors, and windows from floor plans.<br>Convert extracted data into structured JSON format.<br>Generate interactive 3D interior models.<br>Reduce manual effort and improve visualization accuracy. |

**Student Signatures**

1.
2.

**Signature of the Guide**

# ACKNOWLEDGEMENT

First and foremost, I sincerely salute our esteemed institution **V.R SIDDHARTHA ENGINEERING COLLEGE** for giving me this opportunity for fulfilling my project. I am grateful to our Principal **Dr. A.V. RATNA PRASAD**, for his encouragement and support all through the way of my project.

On the submission of this Project report, I would like to extend my honor to **Dr. M. Suneetha**, **Ph.D., Dean R&D, Professor & Head of the Department,** IT for her constant motivation and support during the course of my work.

I feel glad to express my deep sense of gratefulness to my project guide **Dr. Y .Sandeep, Sr.Assistant Professor** for **his** guidanceand assistance in completing this project successfully.

I would also like to convey my sincere indebtedness to all faculty members, including supporting staff of the Department, friends and family members who bestowed their great effort and guidance at appropriate times without which it would have been very difficult on my part to finish the project work.

# TABLE OF CONTENTS


| | |
|---|---|
| ACKNOWLEDGEMENT | iv |
| LIST OF FIGURES | vi |
| LIST OF TABLES | vii |
| LIST OF EQUATIONS | viii |
| ABSTRACT | ix |
| **CHAPTER-1 Introduction** | 01-06 |
| 1.1 Origin of the Problem | 01 |
| 1.2 Basic Definitions and Background | 02 |
| 1.3 Problem Statement | 04 |
| 1.4 Societal Impact of Proposed Work | 05 |
| 1.5 Applications | 06 |
| **CHAPTER-2 Review of Literature** | 07-22 |
| 2.1 Description of Existing Systems | 07 |
| 2.2 Summary of Literature Study | 16 |
| 2.3 Software Requirement Specification | 33 |
| **CHAPTER-3 Proposed Method** | 23-41 |
| 3.1 Design Methodology | 23 |
| 3.2 Description of Algorithms | 27 |
| 3.3 System Architecture Diagram | 33 |
| 3.4 Summary | 41 |
| **CHAPTER-4 Results and Observations** | 42-59 |
| 4.1 Stepwise description of Results | 42 |
| 4.2 Result analysis and Test case results | 49 |
| 4.3 Observations From The Work | 58 |
| 4.4 Summary | 59 |
| **CHAPTER-5 Conclusion and Future study** | 60-63 |
| 5.1 Conclusion | 60 |
| 5.2 Future Work | 62 |
| REFERENCES | 65 |

# LIST OF FIGURES


| Figure | Description | Page |
|---|---|---|
| Figure 3.1 | Architecture Diagram | 35 |
| Figure 4.1 | Budget Input | 43 |
| Figure 4.2 | Sample AI-Generated Room Design | 44 |
| Figure 4.3 | Furniture Costing and Retailer Links | 44 |
| Figure 4.4 | 2D floor plan to 3D model conversion | 45 |
| Figure 4.5 | Usage of HTTPS and SSL | 46 |
| Figure 4.6 | Real Time AR visualization | 46 |
| Figure 4.7 | Performance Graph | 48 |
| Figure 4.8 | CAD to JSON structure Conversion | 49 |
| Figure 4.9 | JSON to 3D room structure | 49 |
| Figure 4.10 | UI Screen | 50 |
| Figure 4.11 | Floor Plan Editor | 51 |
| Figure 4.12 | Space Utilization graph | 52 |
| Figure 4.13 | Cost analysis graph | 53 |
| Figure 4.14 | Time efficiency and Time Taken | 54 |
| Figure 4.15 | Metrics Evaluation of System | 55 |

# LIST OF TABLES


| Table | Description | Page |
|---|---|---|
| Table 2.1 | Literature Review | 19 |
| Table 3.1 | DXF Parsing Algorithm | 27 |
| Table 3.2 | Geometry to JSON | 28 |
| Table 3.3 | Wall Extrusion | 30 |
| Table 3.4 | Rendering | 31 |
| Table 3.5 | Coordinate Mapping | 33 |
| Table 3.6 | Object Placement | 34 |
| Table 4.1 | Geometric Accuracy of 3D Model | 37 |

# LIST OF EQUATIONS


| Equation No | Description | Page |
|---|---|---|
| Eq - (1) | Coordinate Normalization | 36 |
| Eq - (2) | Geometry Parsing Function | 37 |
| Eq - (3) | Corner Intersection Equation | 37 |
| Eq - (4) | 2D to 3D Coordinate Mapping | 37 |
| Eq – (5) | Scaling Transformation | 37 |
| Eq – (6) | Depth Mapping (y → z conversion) | 37 |
| Eq – (7) | Height Assignment | 37 |
| Eq – (8) | Direction Vector Computation | 37 |
| Eq – (9) | Normal Vector Computation | 38 |
| Eq – (10) | Normalization of Vector | 38 |
| Eq – (11) | Wall Thickness Expansion | 38 |
| Eq – (12) | Linear Interpolation (Object Placement) | 38 |
| Eq – (13) | Rotation Angle Calculation | 39 |
| Eq – (14) | Collision Detection Condition | 39 |

# ABSTRACT

This project presents an automated system for converting CAD-based floor plans into structured JSON and generating interactive 3D room visualizations. The system accepts multiple input formats such as DXF, images, and PDF files, and processes them to extract essential architectural elements including walls, corners, doors, and windows. By transforming unstructured design inputs into a standardized Blueprint3D JSON format, the system enables seamless data representation for further visualization and analysis.

The methodology involves parsing CAD files using specialized libraries, along with image-based processing techniques for non-CAD inputs. Extracted geometric features are organized into structured data, where corners define spatial coordinates and walls represent connections between them. Additional components such as furniture, doors, and windows are mapped as items within the JSON structure. This structured approach ensures consistency, scalability, and compatibility with modern visualization tools.

The generated JSON data is then rendered into a 3D environment using Three.js, enabling users to interactively explore the reconstructed interior layout. The system reduces manual effort, improves accuracy in layout interpretation, and provides a user-friendly interface for real-time visualization. This solution has practical applications in architecture, interior design, and real estate, offering an efficient bridge between 2D floor plans and immersive 3D representations.



**KEYWORDS:** CAD Floor Plans, DXF Parsing, Blueprint3D JSON, 3D Visualization, Interior Design Automation, Floor Plan Conversion, Three.js, Architectural Modeling.

# CHAPTER-1

# INTRODUCTION

This chapter introduces the foundation and motivation behind the project, provides the basic definitions and background knowledge necessary to understand the work, outlines the problem statement, discusses the societal impact of the proposed work, and concludes with a summary.

## 1.1 ORGIN OF PROBLEM

In the field of architecture and interior design, floor plans are primarily created in 2D formats such as CAD drawings. While these plans effectively represent structural layouts, they lack depth and do not provide a clear understanding of spatial relationships. As a result, designers, clients, and stakeholders often find it difficult to visualize how the final space will look in reality. This gap between 2D representation and 3D perception creates a major challenge in design communication and decision-making.

To overcome this limitation, professionals typically convert 2D floor plans into 3D models using specialized software tools. However, this process is largely manual and involves recreating walls, placing objects, defining dimensions, and applying textures. Such manual modeling is not only time-consuming but also requires skilled expertise in 3D design tools. For large or complex layouts, the process becomes even more tedious, increasing the chances of errors and inconsistencies in the final model.

Another significant issue is the lack of standardization between different formats. CAD files, images, and PDFs all store spatial data differently, making it difficult to directly extract meaningful structural information. This results in additional preprocessing efforts before any 3D modeling can begin. Furthermore, frequent design modifications require repeated manual updates in the 3D model, leading to inefficiencies in the workflow.

With the increasing demand for faster project delivery and real-time visualization, especially in areas like real estate and interior design, there is a strong need for automated solutions. Existing tools either focus only on CAD visualization or provide limited automation without complete end-to-end integration. This creates a gap for a system that can directly process floor plan inputs, extract essential features, and generate accurate 3D models without extensive manual intervention.

The origin of this problem lies in the absence of an integrated pipeline that connects 2D design inputs with 3D visualization outputs. This project addresses this gap by introducing a system that automates the conversion of CAD-based floor plans into structured JSON and renders them into interactive 3D environments, thereby reducing effort, improving accuracy, and enhancing user experience.

## 1.2 BASIC DEFINITIONS AND BACKGROUND

### 1.2.1 Computer-Aided Design (CAD):

Computer-Aided Design (CAD) refers to the use of computer software to create precise drawings and technical representations of buildings and structures. CAD files, such as DXF, store geometric information including lines, arcs, and shapes that define walls, doors, and other structural elements. These files are widely used in architecture and engineering for designing floor plans. CAD provides high accuracy and scalability but primarily represents layouts in 2D format. This creates a need for further processing to enable 3D visualization.

### 1.2.2 DXF (Drawing Exchange Format):

DXF is a file format developed for enabling interoperability between different CAD software applications. It stores drawing data in a structured format consisting of entities such as lines, polylines, circles, and text. These entities represent various components of a floor plan like walls, openings, and annotations. DXF files are easy to parse programmatically, making them suitable for automated processing systems. They serve as the primary input for extracting geometric and spatial information in this project.

### 1.2.3 Geometry Extraction and Feature Detection:

Geometry extraction is the process of identifying structural elements such as walls, corners, doors, and windows from input files. In CAD files, walls are typically represented as line segments, while intersections of lines indicate corners. Feature detection involves analyzing these geometric primitives and converting them into meaningful architectural components. This step is crucial for transforming raw input data into structured representations. Accurate extraction ensures correctness in the final 3D visualization.

### 1.2.4 Structured JSON Representation (Blueprint3D):

JSON (JavaScript Object Notation) is a lightweight data format used for storing and exchanging structured information. In this project, the extracted geometry is converted into a Blueprint3D-compatible JSON format. This includes defining corners with coordinates, walls as connections between corners, and items such as doors, windows, and furniture. The structured representation ensures consistency and easy integration with visualization systems. It acts as a bridge between 2D data and 3D rendering.

### 1.2.5 Floor Plan to 3D Conversion Pipeline:

The floor plan conversion pipeline refers to the sequence of steps involved in transforming input data into a 3D model. It begins with file upload, followed by parsing and geometry extraction. The extracted features are then structured into JSON format and passed to the

rendering engine. Each stage plays a critical role in ensuring accuracy and efficiency. The pipeline automates the entire process, reducing manual effort and improving workflow.

### 1.2.6 3D Rendering using Three.js:

3D rendering is the process of generating a visual representation of a space using graphical techniques. Three.js is a JavaScript library that enables rendering of 3D models directly in the browser using WebGL. It allows the creation of scenes, cameras, lighting, and textures to produce realistic environments. In this project, the JSON data is used to construct walls, floors, and objects in a 3D space. This enables users to interactively explore the generated room layout.

### 1.2.7 Evaluation Metrics:

To evaluate the performance of the system, the following metrics are considered:

**Geometric Accuracy:**

Geometric accuracy measures how precisely the extracted walls and corners match the original floor plan. It evaluates whether spatial dimensions and alignments are correctly preserved. High accuracy ensures reliable 3D reconstruction. This metric is essential for validating structural correctness.

**Conversion Efficiency:**

Conversion efficiency measures the time taken to process input files and generate the final JSON output. It reflects the system's ability to handle data quickly and effectively. Higher efficiency indicates better performance and scalability. This is important for real-time applications.

**Visualization Quality:**

Visualization quality evaluates how realistic and interactive the generated 3D model is. It considers factors such as rendering clarity, lighting, and object placement. A higher quality improves user experience and understanding of the space. This metric reflects the effectiveness of the rendering system.

**Data Consistency:**

Data consistency measures how well the structured JSON maintains relationships between elements such as walls, corners, and items. It ensures there are no missing or conflicting connections. High consistency leads to stable and error-free 3D models. This is important for system reliability.

**Scalability:**

Scalability evaluates the system's ability to handle complex and large floor plans. It checks whether performance remains stable with increasing input size. A scalable system can process real-world architectural layouts efficiently. This ensures practical usability.

## 1.3 PROBLEM STATEMENT

In the domains of architecture, interior design, and real estate, floor plans are primarily created using Computer-Aided Design (CAD) tools in 2D formats such as DXF. While these 2D representations are precise and widely used by professionals, they fail to provide an intuitive understanding of spatial depth, proportions, and real-world appearance. For clients, stakeholders, and even non-specialist designers, interpreting 2D layouts can be challenging, often leading to confusion in understanding room dimensions, object placement, and overall design flow.

To bridge this gap, 3D visualization is commonly used. However, the process of converting 2D floor plans into 3D models is largely manual and involves recreating structural elements such as walls, doors, windows, and furniture using specialized 3D modeling software. This manual reconstruction requires significant time, effort, and expertise. For complex layouts, the process becomes even more tedious, increasing the likelihood of human errors such as incorrect dimensions, misaligned walls, or improper placement of objects. Additionally, any modification in the original floor plan requires repeated manual updates in the 3D model, further reducing efficiency.

Another major challenge lies in the diversity of input formats. Floor plans may exist in multiple formats such as CAD files (DXF), images (JPG/PNG), or PDF documents. Each format represents spatial and structural information differently, making it difficult to extract meaningful data directly. Existing tools either support limited formats or lack the ability to perform complete end-to-end conversion. As a result, users often rely on multiple tools and manual interventions, leading to fragmented workflows.

Furthermore, current solutions do not provide a unified pipeline that seamlessly integrates data extraction, structured representation, and real-time visualization. While some systems focus on CAD parsing and others on 3D rendering, there is a lack of systems that combine both functionalities effectively. This creates a gap where users cannot directly convert floor plans into interactive 3D environments without extensive preprocessing and manual modeling.

With the increasing demand for faster project delivery, real-time visualization, and user-friendly design tools, especially in fields like interior design and real estate marketing, there is a strong

need for an automated and integrated solution. Therefore, this project aims to address these challenges by developing a system that can process various floor plan formats, extract structural elements, convert them into a standardized JSON representation, and generate interactive 3D visualizations. This approach reduces manual effort, improves accuracy, and enhances the overall efficiency of the design and visualization process.

### 1.3.1 Objectives:

- To develop an automated system for converting CAD-based floor plans into structured data
- To extract key architectural elements such as walls, corners, doors, and windows
- To support multiple input formats including DXF, images, and PDF files
- To generate a standardized JSON format for representing floor plan data
- To render interactive 3D visualizations using Three.js
- To reduce manual effort and improve efficiency in 3D model generation

### 1.3.2 Outcomes:

- Successful extraction of geometric and structural information from input files
- Generation of accurate Blueprint3D JSON representations
- Interactive 3D visualization of interior layouts
- Reduced time and effort compared to manual modelling
- Improved accuracy and consistency in layout representation
- A scalable system capable of handling different floor plan formats

## 1.4 SOCIETAL IMPACT OF PROPOSED WORK

**1.4.1** This project has significant societal impact by simplifying the design and visualization process for a wide range of users. It makes architectural designs more accessible and understandable, especially for clients who may not have technical expertise in reading 2D floor plans. By enabling clear 3D visualization, it improves communication between designers and clients, reducing misunderstandings and enhancing decision-making.

**1.4.2** The system also contributes to increased productivity in the architecture and interior design industries by automating repetitive tasks. This allows professionals to focus more on creativity and innovation rather than manual modeling. Additionally, it can support educational purposes by helping students better understand spatial concepts through interactive 3D representations.

**1.4.3** In the long term, such automation can lead to smarter design systems and integration with advanced technologies like augmented reality and virtual reality, further enhancing user experience and accessibility.

## 1.5 APPLICATIONS

- **1.5.1 Architecture:** Automated conversion of floor plans into 3D models for design visualization
- **1.5.2 Interior Design:** Planning and previewing room layouts with furniture placement
- **1.5.3 Real Estate:** Showcasing properties with interactive 3D views for buyers
- **1.5.4 Construction:** Assisting in layout understanding and planning before execution
- **1.5.5 Smart Home Design:** Integration with digital platforms for designing intelligent living spaces
- **1.5.6 AR/VR Systems:** Extending the project for immersive visualization experiences

## 1.6 SUMMARY

This chapter introduces the concept of converting CAD-based floor plans into interactive 3D visualizations. It begins by explaining the origin of the problem, highlighting the limitations of 2D floor plans in representing spatial depth and the challenges involved in manually creating 3D models. The need for an automated solution to improve efficiency and accuracy in architectural visualization is emphasized.

The chapter further provides basic definitions and background of key concepts such as CAD files, DXF format, geometry extraction, JSON representation, and 3D rendering using Three.js. These concepts form the foundation for understanding how the system processes input data and transforms it into structured and visual outputs.

Additionally, the problem statement clearly defines the challenges of manual conversion, lack of integration, and inefficiencies in current workflows. Finally, the chapter discusses various applications of the proposed system in architecture, interior design, real estate, and education, demonstrating its practical relevance and impact.

# CHAPTER – 2

# REVIEW OF LITERATURE

This chapter describes the review of literature that we have taken from various papers and considered all the points mentioned in the papers.

## 2.1 DESCRIPTION OF EXISTING SYSTEM

This section presents a comprehensive and in-depth analysis of existing systems and research works related to Computer-Aided Design (CAD) processing, 3D reconstruction, and interior visualization technologies. The purpose of this analysis is to examine how current methodologies handle the transformation of raw architectural data into meaningful, structured, and visually interactive representations.

In modern architectural and interior design workflows, CAD drawings serve as the primary source of spatial and geometric information. These drawings contain detailed representations of building layouts, including walls, doors, windows, furniture placement, and other structural elements. However, despite their richness in detail, CAD files are typically composed of low-level geometric primitives such as lines, arcs, circles, and polylines, which lack semantic meaning and structured organization. This unstructured nature makes it challenging to directly utilize CAD data for advanced applications such as automated 3D reconstruction, simulation, and interactive visualization.

To address these challenges, several research efforts have focused on extracting geometric information from CAD drawings and converting it into structured formats. Structured data representations such as JSON and GeoJSON play a crucial role in organizing spatial information into hierarchical and machine-readable forms. These formats enable efficient storage, retrieval, and processing of architectural data, while also improving interoperability between different systems and platforms. By transforming raw geometric entities into structured objects, these approaches simplify downstream tasks such as feature extraction, object recognition, and 3D model generation.

In addition to data structuring, significant advancements have been made in the field of intelligent CAD processing using machine learning and deep learning techniques. These approaches aim to automate the interpretation of CAD data by identifying patterns, recognizing structural components, and understanding spatial relationships. Techniques such as sequence modeling and feature-based learning have improved the accuracy and efficiency of CAD parsing, reducing the need for manual intervention. However, these methods often require large datasets and high computational resources, which can limit their practical applicability in real-time systems.

Another important area of research is 3D reconstruction, which involves converting 2D floor plans

or CAD drawings into three-dimensional models. This process requires accurate identification of structural elements, preservation of spatial relationships, and generation of realistic geometry. Various approaches have been proposed, including feature matching, geometric transformation, and AI-based reconstruction methods. While these techniques have shown promising results, many of them are limited in scope and do not provide a complete automated solution for full-scale architectural layouts.

Furthermore, visualization technologies such as augmented reality (AR) and interactive 3D modeling have significantly enhanced the way users interact with architectural designs. These systems allow users to explore virtual environments, visualize interior layouts, and make design decisions in real time. AR-based applications, in particular, provide immersive experiences by overlaying digital models onto real-world environments. However, most of these systems rely on pre-existing 3D models and do not include automated pipelines for generating models directly from CAD data.

Artificial intelligence has also emerged as a powerful tool in interior design, enabling automated layout generation, design recommendations, and workflow optimization. AI-driven systems can analyze user preferences, predict optimal configurations, and improve overall design efficiency. Despite these advancements, most AI-based solutions focus on design assistance rather than integrating CAD data processing, structured data generation, and 3D visualization into a unified system.

Overall, the existing systems demonstrate significant progress in individual areas such as data structuring, CAD interpretation, 3D reconstruction, visualization, and AI-based design. However, these approaches are often fragmented and address only specific components of the problem. There is a lack of a comprehensive, end-to-end system that seamlessly integrates CAD data extraction, structured representation, intelligent processing, and real-time 3D visualization.

This limitation highlights a clear research gap and emphasizes the need for a unified solution that combines all these functionalities into a single, efficient pipeline. The proposed system aims to address this gap by developing an integrated framework that transforms CAD floor plans into structured data and generates interactive 3D visualizations, thereby improving automation, accuracy, and user experience.

**Standardization of CAD Drawing Formats and GeoJSON-Based Processing for 3D Spatial Data Extraction**

**Authors:** Lee, Jongseo; Yudoo Kim; Il-Young Moon

**Year:** 2024

**Observations:**

This paper addresses one of the fundamental challenges in CAD-based systems, which is the lack of standardization in CAD drawing formats. CAD files typically contain raw geometric entities such as lines, arcs, circles, splines, and polygons, but these elements are often stored in proprietary or loosely structured formats. This makes it difficult to directly extract meaningful spatial information for further processing such as 3D modeling or visualization.

The authors propose a systematic approach to convert CAD data into GeoJSON, a widely accepted structured data format used for representing spatial information. By transforming geometric primitives into structured objects, the system enables better organization of spatial relationships such as adjacency, connectivity, and boundaries. This structured representation significantly simplifies downstream operations, including feature extraction, segmentation, and mapping.

Another important contribution of this work is improved interoperability. Since GeoJSON is compatible with many modern web-based visualization frameworks and GIS systems, the converted data can be easily integrated into different platforms. This enhances flexibility and scalability in system design.

The study also demonstrates that structured formats reduce computational complexity during processing. Instead of repeatedly interpreting raw CAD geometry, systems can directly access organized data, leading to faster and more efficient processing pipelines.

However, despite these advantages, the approach is limited to data conversion and preprocessing. It does not include mechanisms for semantic understanding, intelligent feature recognition, or automatic 3D reconstruction. Additionally, it lacks integration with rendering engines or interactive visualization systems, making it incomplete for end-to-end applications.[1]

**JSON Application Programming Interface for Data Exchange**

**Author:** Papagiannopoulos, Ioannis

**Year:** 2015

**Observations:**

This work focuses on JSON as a universal data interchange format and highlights its importance in modern software architectures. JSON is particularly useful due to its lightweight syntax, ease of parsing, and ability to represent hierarchical relationships, which are essential for modeling complex systems such as architectural layouts.

The paper explains how JSON enables modular system design by allowing different components (such as data extraction modules, processing engines, and visualization interfaces) to communicate

efficiently. In the context of CAD and interior design systems, JSON can be used to store structured representations of floor plans, including room dimensions, object placements, and connectivity relationships.

Another key advantage discussed is scalability. JSON structures can easily accommodate additional attributes such as material properties, object metadata, and semantic labels without requiring major changes to the system architecture. This flexibility makes it suitable for evolving systems that require continuous updates.

The study also emphasizes the role of JSON in web-based visualization systems, where data needs to be transmitted between servers and clients efficiently. Its compatibility with JavaScript frameworks further enhances its usability in interactive applications.

However, the limitation of this work is that it does not provide domain-specific solutions for CAD data processing. It serves as a foundational technology rather than a complete system. It does not address how raw CAD data should be converted into JSON or how such structured data can be used for automated 3D reconstruction.[2]

**CADParser: A Learning Approach of Sequence Modeling for B-Rep CAD**

**Authors:** Zhou, Shengdi; Tianyi Tang; Bin Zhou

**Year:** 2023

**Observations:**

This research introduces a deep learning-based framework for interpreting CAD models using sequence modeling techniques. Unlike traditional rule-based systems, this approach treats CAD construction as a sequence of operations, similar to a program. By learning patterns from large datasets, the model can reconstruct geometric structures and understand design intent.

The system is capable of identifying complex geometric features such as edges, faces, surfaces, and topological relationships. This significantly improves the accuracy of CAD interpretation compared to manual or heuristic-based methods. It also reduces dependency on handcrafted rules, making the system more adaptable to different CAD formats.

Another important contribution is automation. The model can automatically parse CAD data and generate structured representations, which can be used for further processing such as simulation or visualization.

However, the approach has several limitations. It requires large annotated datasets for training, which may not always be available. The computational cost of training and inference is also high, making it unsuitable for real-time applications. Furthermore, the system focuses primarily on understanding CAD models rather than integrating with visualization pipelines or generating

complete 3D environments. [3]

**Tiny-CAD-Coder: Leveraging Pre-trained Code Models for CAD Generation**

**Authors:** Berger, Elias, et al.

**Observations:**

This paper explores the use of transformer-based models for CAD generation by leveraging pre-trained code models. The system treats CAD design as a form of programming, where geometric structures are generated through sequences of commands.

The use of pre-trained models allows the system to learn complex design patterns and generate CAD structures efficiently. This significantly reduces manual effort and improves productivity in design workflows. The approach also demonstrates the potential of combining artificial intelligence with CAD systems to enable intelligent design automation.

Another advantage is adaptability. The model can generalize across different design scenarios and generate diverse structures based on input conditions.

However, the system is primarily focused on generating new CAD designs rather than processing existing ones. It does not address the problem of extracting features from 2D floor plans or converting them into structured data. Additionally, it lacks integration with visualization systems, limiting its applicability in end-to-end solutions. [4]

**Seeing 3D Chairs: Exemplar Part-Based 2D–3D Alignment Using CAD Models**

**Authors:** Aubry, Mathieu, et al.

**Year:** 2014

**Observations:**

This paper presents a method for aligning 2D images with 3D CAD models using part-based matching techniques. The system identifies key features in 2D images and maps them to corresponding components in 3D models, ensuring accurate alignment.

The approach preserves spatial relationships and improves reconstruction accuracy by focusing on individual parts rather than entire objects. This makes it effective for recognizing complex structures and maintaining geometric consistency.

The study highlights the importance of feature matching in 2D-to-3D conversion and demonstrates how accurate mapping can enhance reconstruction quality.

However, the method is limited in scope as it is designed for specific object categories such as chairs. It does not generalize well to large-scale architectural layouts or floor plans. Additionally, it does not provide a complete pipeline for converting CAD drawings into 3D environments.[5]

**A Deep Learning Approach to the Classification of 3D CAD Models**

**Authors:** Qin, Fei-wei, et al.

**Year:** 2014

**Observations:**

This study focuses on the classification of 3D CAD models using deep learning techniques, which represents an important step toward intelligent understanding of geometric data. The system extracts geometric and topological features from CAD models and uses them as input for deep neural networks to categorize models into different predefined classes. These features may include shape descriptors, surface characteristics, curvature information, and spatial relationships between components.

One of the key contributions of this research is its emphasis on feature representation. Instead of relying on manual feature engineering, the system leverages deep learning to automatically learn meaningful representations from data. This significantly improves classification accuracy and allows the model to generalize across different CAD structures. The approach also enhances the organization and retrieval of CAD models in large databases, making it useful for applications such as model search, reuse, and design recommendation.

Furthermore, the study demonstrates that deep learning can capture complex geometric patterns that traditional methods may fail to recognize. This leads to improved performance in identifying structural similarities and differences between models.

However, the approach is limited to classification tasks and does not extend to other critical areas such as geometric reconstruction, semantic interpretation, or visualization. It does not provide mechanisms for converting 2D floor plans into 3D models, which is essential for interior design systems. Additionally, it lacks integration with real-time rendering or interactive visualization frameworks, limiting its applicability in complete design pipelines.[6]

**Architectural Visualization using Augmented Reality**

**Authors:** Raj, Nikhil; Hema P. Menon; Abhinand Satheesh

**Year:** 2025

**Observations:**

This paper explores the application of augmented reality (AR) in architectural visualization, focusing on enhancing user interaction and spatial understanding. The system enables users to visualize 3D architectural models in real-world environments using mobile devices such as smartphones or tablets. By overlaying digital models onto physical spaces, the system provides an immersive and intuitive experience.

A major advantage of this approach is real-time interaction. Users can navigate through the environment, adjust viewpoints, and explore different design perspectives dynamically. This improves decision-making by allowing users to evaluate designs before actual construction. The system also enhances client engagement by providing a more realistic representation of architectural concepts.

Another important contribution is improved spatial perception. AR allows users to understand scale, proportions, and spatial relationships more effectively compared to traditional 2D drawings or static 3D models.

However, the system heavily depends on pre-built 3D models, which must be manually created using external tools. It does not include automated mechanisms for generating these models from CAD drawings. Additionally, challenges such as device limitations, tracking accuracy, and environmental constraints may affect performance. As a result, the system is more suitable for visualization rather than full automation of the design process.[7]

**Interactive Interior Design Using Augmented Reality and 3D Modeling**

**Authors:** Sharika, T. R., et al.

**Year:** 2024

**Observations:**

This research presents an interactive interior design system that integrates augmented reality with 3D modeling to provide a dynamic and user-friendly design environment. The system allows users to place, move, and modify interior elements such as furniture, walls, and decorative items in real time.

One of the key strengths of this approach is its focus on user interaction. The system enables intuitive manipulation of objects through gestures or touch-based controls, making it accessible even to non-expert users. It also supports real-time feedback, allowing users to instantly visualize changes and experiment with different design configurations.

The integration of AR enhances realism by placing virtual objects within real-world environments. This helps users better understand spatial arrangements, object proportions, and overall aesthetics. The system also supports customization, enabling users to personalize designs based on preferences.

However, the system does not address the automatic extraction of structural data from CAD floor plans. All models and layouts must be manually defined or pre-loaded, which limits automation. Additionally, it lacks advanced features such as semantic understanding of spaces or intelligent layout generation.[8]

**Visualization of Furniture Model using Augmented Reality**

**Authors:** Vaidya, Gendlal M., et al.

**Year:** 2022

**Observations:**

This paper focuses on the use of augmented reality for visualizing furniture models in real-world environments. The system allows users to place virtual furniture objects within a physical space and evaluate their placement, size, and appearance.

A significant advantage of this approach is improved decision-making. Users can preview how furniture will look and fit within a space before purchasing or implementing it. This reduces the risk of incorrect sizing or poor design choices.

The system also enhances spatial understanding by accurately representing scale and positioning. It enables users to experiment with different arrangements and configurations, improving overall design quality.

However, the system is limited to object-level visualization and does not include broader architectural elements such as walls, rooms, or structural layouts. It does not support floor plan processing or 3D reconstruction from CAD data. Additionally, it lacks integration with automated design systems, making it a standalone visualization tool rather than a comprehensive solution.[9]

**Optimizing AI's Role in Advancing Interior Design Industry**

**Authors:** Lesmana, V. A. A., et al.

**Year:** 2024

**Observations:**

This study explores the role of artificial intelligence in transforming the interior design industry by automating various aspects of the design process. AI techniques are used for tasks such as layout generation, cost estimation, material selection, and design optimization.

One of the key contributions is the ability of AI systems to analyze large datasets and generate intelligent design recommendations. These systems can consider multiple factors such as user preferences, space constraints, and design trends to produce optimized layouts.

The study also highlights improvements in efficiency and productivity. By automating repetitive tasks, AI reduces the workload on designers and accelerates the design process. It also enables personalized design solutions tailored to individual user needs.

However, the approach does not integrate CAD data processing or visualization systems. It operates primarily at a conceptual level and lacks mechanisms for converting CAD drawings into structured data or 3D models. This limits its applicability in fully automated design pipelines.[10]

**Application of AI Technology in Interior Design**

**Authors:** Chen, Z.; Wang, X.

**Year:** 2020

**Observations:**

This work examines the application of artificial intelligence in interior design, focusing on automation, personalization, and efficiency. AI-driven systems can generate layout suggestions, recommend color schemes, and optimize space utilization based on user preferences.

The study emphasizes the role of AI in enhancing creativity and reducing manual effort. By analyzing design patterns and user behavior, AI systems can provide intelligent recommendations that improve design quality.

Another important contribution is improved decision-making. AI tools can evaluate multiple design alternatives and suggest optimal solutions based on predefined criteria.

However, the study remains largely conceptual and does not provide a detailed implementation framework. It does not address integration with CAD systems or the conversion of floor plans into 3D models. As a result, its practical applicability in automated design systems is limited.[11]

**Application and Practice of Artificial Intelligence Technology in Interior Design**

**Author:** Gong, M.

**Year:** 2023

**Observations:**

This paper focuses on practical implementations of AI in interior design workflows. It demonstrates how intelligent algorithms can assist in tasks such as layout planning, color matching, and space optimization.

The study highlights improvements in productivity and design accuracy. AI systems can automate repetitive tasks and provide data-driven insights, enabling designers to focus on creative aspects.

It also supports customization, allowing designs to be tailored to specific user requirements. This enhances user satisfaction and overall design effectiveness.

However, the research does not address CAD data processing or automated 3D model generation. It lacks integration with visualization systems and does not provide a complete end-to-end solution.[12]

**Improving Design Efficiency Using Artificial Intelligence**

**Authors:** Samar Noaman; G. Hussein

**Year:** 2023

**Observations:**

This research investigates how AI can improve efficiency in the interior design process by streamlining various stages such as concept development, layout planning, and visualization

The study demonstrates that AI tools can significantly reduce design time and improve workflow productivity. By automating routine tasks, designers can focus on more complex and creative aspects of design.

AI systems also enhance decision-making by analyzing large datasets and identifying optimal solutions. This leads to better design outcomes and improved user satisfaction.

However, the study does not focus on CAD data extraction or 3D reconstruction. It lacks integration with systems that convert 2D floor plans into interactive 3D environments, limiting its applicability in automated design pipelines.[13]

**PropARt: Augmented Reality-Enabled Multi-Furniture Placement Application**

**Authors**: Harihara; Sreenath Panicker; et al.

**Year:** 2024

**Observations:**

This paper presents an augmented reality-based system for placing multiple furniture items in real-world environments. The application allows users to visualize and arrange furniture interactively, improving spatial understanding and design flexibility.

A key strength of the system is its support for multi-object interaction. Users can experiment with different layouts, adjust positions, and evaluate design configurations in real time. This enhances decision-making and user engagement.

The system also improves visualization of scale, alignment, and aesthetics, providing a realistic representation of interior spaces.

However, the approach depends on pre-designed 3D models and does not include automated generation from CAD floor plans. It lacks structured data extraction and integration with CAD processing systems, limiting its automation capabilities and scalability.[14]

## 2.2 SUMMARY OF LITERATURE STUDY

This section presents a comprehensive synthesis of the literature reviewed in the study, highlighting key research contributions, methodologies, technological advancements, and existing limitations in the domains of CAD data processing, structured data representation, 3D reconstruction, visualization technologies, and artificial intelligence in interior design. The

objective of this summary is to consolidate insights from various research works and identify the research gaps that motivate the proposed system.

The literature indicates that one of the primary challenges in CAD-based systems is the unstructured nature of CAD drawings. CAD files typically consist of low-level geometric primitives such as lines, arcs, circles, and polylines, which lack semantic meaning and organized structure. This makes direct processing difficult for applications such as 3D modeling, spatial analysis, and visualization. To address this issue, several studies emphasize the importance of transforming CAD data into structured formats such as JSON and GeoJSON. These structured representations enable hierarchical organization of spatial information, making it easier to extract features such as boundaries, connectivity, and spatial relationships. Furthermore, structured data formats improve interoperability between systems and facilitate efficient communication across different modules, especially in web-based and distributed applications.

In addition to data structuring, significant progress has been made in the application of machine learning and deep learning techniques for CAD interpretation. Learning-based approaches, such as sequence modeling and feature-based classification, enable automated extraction of geometric and topological features from CAD models. These techniques improve accuracy and reduce dependency on manual feature engineering by learning patterns directly from data. They also enhance the ability of systems to understand complex designs and recognize structural components. However, these approaches often require large training datasets, high computational resources, and extensive preprocessing, which can limit their scalability and real-time applicability. Moreover, most of these methods focus on specific tasks such as classification or parsing, rather than providing a complete pipeline for end-to-end processing.

Research in 2D-to-3D reconstruction highlights the importance of accurately converting 2D representations into 3D models while preserving spatial relationships and geometric consistency. Techniques such as feature matching, geometric transformation, and part-based alignment have been proposed to improve reconstruction accuracy. These methods demonstrate that identifying key structural elements and maintaining their relationships is essential for generating realistic 3D models. However, many of these approaches are limited to specific object categories or controlled environments and do not generalize well to complex architectural layouts such as full floor plans.

Visualization technologies, particularly augmented reality (AR) and interactive 3D modeling systems, have significantly enhanced the way users interact with architectural and interior designs. AR-based systems allow users to visualize 3D models in real-world environments, providing immersive experiences and improving spatial understanding. These systems support real-time

interaction, enabling users to explore different design configurations and make informed decisions. Similarly, interactive 3D modeling tools allow dynamic modification of layouts, object placement, and customization of designs. Despite these advancements, most visualization systems rely on pre-built 3D models and do not include automated pipelines for generating models directly from CAD data. This creates a disconnect between data processing and visualization.

Artificial intelligence has emerged as a transformative technology in the interior design domain. AI-driven systems can automate various aspects of the design process, including layout generation, space optimization, material selection, and design recommendation. These systems improve efficiency, reduce manual effort, and enable personalized design solutions based on user preferences. AI also enhances decision-making by analyzing large datasets and identifying optimal design configurations. However, most AI-based approaches focus on design assistance rather than integrating with CAD data processing and 3D reconstruction. They often operate at a conceptual level and lack practical implementation frameworks for end-to-end automation.

Another important observation from the literature is the fragmentation of existing solutions. Most research works address individual components of the problem, such as CAD parsing, structured data representation, AI-based design, or visualization, in isolation. There is a lack of integration between these components, resulting in systems that are incomplete and unable to provide a seamless workflow from input to output. For example, some systems focus on data extraction but do not support visualization, while others provide visualization but rely on manually created models.

Furthermore, existing systems often lack scalability, flexibility, and real-time performance. Many approaches require extensive preprocessing, manual intervention, or high computational resources, which limits their usability in practical applications. There is also a lack of standardization in data formats and processing techniques, which further complicates system integration and interoperability.

From the analysis of the literature, it is evident that there is a significant research gap in developing a unified, end-to-end system that integrates CAD data extraction, structured data conversion, intelligent feature processing, and interactive 3D visualization. Such a system should be capable of automatically converting 2D floor plans into structured representations, generating accurate 3D models, and providing real-time visualization and interaction capabilities.

The proposed system aims to address these limitations by designing a comprehensive pipeline that combines all these functionalities into a single framework. By integrating CAD processing, structured data generation, and visualization technologies, the proposed approach seeks to improve

automation, accuracy, efficiency, and user experience in interior design and architectural applications.

***Table 2.1: Literature Review***

| S.No | Title and Publication Details | Algorithms / Technologies Used | Dataset | Purpose | Remarks |
|---|---|---|---|---|---|
| 1 | Year: 2024 Standardization of CAD drawing formats and GeoJSON-based processing | CAD standardization, GeoJSON conversion | CAD drawings | To convert CAD data into structured formats for 3D extraction | Enhances interoperability and structured data representation |
| 2 | Year: 2015 JSON API for data exchange | JSON representation, Data structuring | Simulation data | To enable efficient structured data exchange | JSON supports hierarchical and spatial data handling |
| 3 | Year: 2023 CADParser: Learning approach for CAD | Deep learning, Sequence modeling | B-Rep CAD models | To automate CAD parsing and structure extraction | Improves accuracy but requires high computational resources |
| 4 | Tiny-CAD-Coder | Transformer models, Pre-trained AI models | CAD datasets | To generate and interpret CAD designs using AI | Reduces manual modeling effort |
| 5 | Year: 2014 Seeing 3D Chairs | 2D–3D alignment, Feature matching | CAD model datasets | To align 2D images with 3D models | Supports accurate reconstruction but limited to objects |
| 6 | Year: 2014 Deep Learning for 3D CAD Classification | Deep learning, Feature extraction | 3D CAD models | To classify and analyze CAD structures | Emphasizes importance of geometric features |
| 7 | Year: 2025 Architectural Visualization using AR | Augmented Reality, 3D rendering | Architectural models | To visualize designs in real-world environments | Enhances user experience but needs pre-built models |

| | | | | | |
|---|---|---|---|---|---|
| 8 | Year: 2024 Interactive Interior Design using AR | AR, 3D modeling | Interior layouts | To enable real-time interior visualization | Improves interaction and customization |
| 9 | Year: 2022 Furniture Visualization using AR | AR visualization | Furniture datasets | To place virtual furniture in real environments | Helps in spatial planning and decision-making |
| 10 | Year: 2024 AI in Interior Design Industry | AI-based optimization | Interior design data | To automate layout generation and design suggestions | Improves efficiency and reduces manual effort |
| 11 | Year: 2020 Application of AI in Interior Design | Artificial Intelligence, Design automation | Interior design datasets | To assist in layout generation and visualization | Focuses on conceptual AI integration |
| 12 | Year: 2023 AI Technology in Interior Design Practice | Intelligent algorithms, AI-based systems | Design case data | To improve workflow efficiency and design accuracy | Provides practical AI applications but lacks CAD integration |
| 13 | Year: 2023 Improving Design Efficiency using AI | AI-based analysis, Automation tools | Interior design workflows | To streamline design processes and reduce time | Enhances productivity but lacks 3D reconstruction focus |
| 14 | Year: 2024 PropARt: AR-based Furniture Placement | Augmented Reality, Multi-object placement | Furniture and AR datasets | To enable interactive furniture placement in real space | Improves visualization but not automated from CAD |

## 2.3 SOFTWARE REQUIREMENT SPECIFICATION

### 2.3.1 Purpose

The purpose of this project is to develop an automated system that converts CAD-based floor plans into structured JSON data and generates interactive 3D interior layouts. The system focuses on extracting structural elements such as walls, corners, doors, and windows, and transforming them

into a format suitable for 3D visualization. It aims to reduce manual effort, improve accuracy, and provide better understanding of spatial layouts using modern rendering technologies.

### 2.3.2 Scope

The system allows users to upload floor plan files in formats such as DXF, images, or PDFs. It processes the input to extract geometric and structural components, converts them into a structured JSON format, and generates a 3D model using rendering tools like Three.js. The system supports interactive features such as zoom, rotation, and navigation, enabling users to explore the generated layout. It is designed to handle both simple and moderately complex floor plans efficiently.

### 2.3.3 Intended Audience

Architects and interior designers

Real estate professionals

Students and researchers

Software developers working on visualization systems

### 2.3.4 Functional Requirements

FR1: The system shall allow users to upload floor plan files (DXF, image, PDF).

FR2: The system shall extract structural elements such as walls, corners, doors, and windows.

FR3: The system shall convert extracted data into structured JSON format.

FR4: The system shall generate 3D models from the structured data.

FR5: The system shall provide interactive visualization features (zoom, rotate, pan).

### 2.3.5 Non-Functional Requirements

NFR1: The system should process input efficiently with minimal delay.

NFR2: The system should maintain accuracy in geometry extraction.

NFR3:The visualization should be smooth and interactive.

NFR4: The system should be scalable for handling complex floor plans.

## 2.4 SUMMARY

This chapter presented a review of existing literature related to CAD data processing, structured representation, 3D reconstruction, and visualization techniques. The reviewed works highlight the importance of converting unstructured CAD data into standardized formats such as JSON for efficient processing and interoperability.

Research studies also emphasize the role of machine learning, feature extraction, and 2D-to-3D conversion methods in improving the accuracy of model generation. Additionally, augmented reality and interactive visualization systems enhance user experience by providing realistic and dynamic representations of interior layouts.

However, most existing systems focus on individual components such as CAD parsing, 3D modeling, or visualization, rather than providing a complete integrated solution. Therefore, there is a need for an end-to-end system that combines data extraction, structured representation, and interactive 3D visualization, which is addressed in the proposed project.

# CHAPTER-3
# PROPOSED METHODOLOGY

This chapter presents the methodology adopted for the proposed system that converts CAD-based floor plans into structured JSON and generates interactive 3D visualizations. It describes the datasets and input formats used, along with their characteristics. The chapter also explains the preprocessing techniques applied to prepare the data, including geometry extraction, coordinate normalization, and feature mapping. The overall pipeline ensures efficient transformation from 2D floor plans to 3D representations.

## 3.1 DESIGN METHODOLOGY

This section explains the design methodology of the proposed system, which focuses on building an integrated framework for automated floor plan processing and 3D visualization. The system is designed to handle multiple input formats such as CAD (DXF), images, and PDF files. It involves extracting geometric features like walls, corners, doors, and windows, and converting them into a structured JSON format compatible with 3D rendering systems.

The methodology consists of several stages including file input, preprocessing, geometry extraction, JSON conversion, and visualization. The system uses parsing techniques (such as DXF parsing using `ezdxf`) to read CAD files and extract entities. For image-based inputs, basic image processing techniques can be applied to detect edges and contours. The extracted features are then mapped into a structured representation where spatial relationships are preserved.

Finally, the structured JSON is passed to a rendering engine implemented using Three.js, where walls, floors, and objects are constructed in a 3D environment. This integrated pipeline ensures automation, accuracy, and real-time visualization.

### 3.1.1 Dataset Description:

The proposed system does not rely on a fixed dataset but instead operates on real-world floor plan inputs. However, different types of input data are used to evaluate the system's effectiveness.

- CAD-Based Data (DXF Files)

  DXF files serve as the primary input source due to their structured representation of geometric entities. These files contain detailed architectural information such as walls, dimensions, and layout structure. Each entity in a DXF file is defined using coordinates, making it suitable for accurate extraction and conversion.

- Image-Based Floor Plans

  Image inputs such as JPG and PNG files are used to simulate scenarios where CAD files are not available. These images may contain scanned floor plans or hand-drawn layouts. Although less precise than CAD data, they provide flexibility in real-world applications.

- PDF Floor Plans

  Type equation here.PDF documents are commonly used to share architectural designs. These are converted into images before processing. The system treats them similarly to image inputs for feature extraction.The diversity of input formats ensures that the system is robust and adaptable to different real-world scenarios.

### 3.1.1 Data Pre-Processing:

Preprocessing plays a crucial role in preparing input data for further processing. It ensures consistency, removes noise, and converts data into a suitable format.

#### 3.1.1.1 CAD Parsing and Coordinate Normalization

For CAD inputs, the DXF file is parsed to extract geometric entities. Let the input file be represented as:

$$\boldsymbol{F}$$

The parsing function extracts entities:

$$\boldsymbol{G} = \boldsymbol{Parse}(\boldsymbol{F})$$

Where **G** represents a set of geometric elements such as lines and polylines.

To convert coordinates into a uniform scale, normalization is applied:

$$(\mathbf{x}', \mathbf{y}') = (\mathbf{x} \times \mathbf{s},\ \mathbf{y} \times \mathbf{s})$$

Where

- $x, y$ = original coordinates
- $s$ = scale factor
- $x', y'$ = normalized coordinates

This step ensures compatibility between different input sizes and units.

#### 3.1.1.2 Geometry Extraction:

The extracted geometric entities are processed to identify structural components.

- Lines → Walls
- Intersections → Corners
- Patterns → Doors and Windows

Let the set of lines be:

$$\mathbf{L} = \{\mathbf{l1}, \mathbf{l2}, \dots, \mathbf{ln}\}$$

Corners are computed as:

$$\mathbf{C} = \{\mathbf{Intersection}(\mathbf{li}, \mathbf{lj})\}$$

Walls are defined as connections between corners:

$$\mathbf{W} = \{(\mathbf{ci}, \mathbf{cj})\}$$

This step is critical for preserving spatial relationships in the floor plan.

#### 3.1.1.3 Image Processing for Non-CAD Inputs

For image-based inputs, preprocessing includes:

- Conversion to grayscale
- Edge detection
- Contour extraction

Let the input image be:

$$\boldsymbol{I}$$

The processed image is:

$$\mathbf{I}' = \mathbf{EdgeDetect}(\mathbf{GrayScale}(\mathbf{I}))$$

Contours extracted from $\mathrm{I}'$ are used to approximate walls and boundaries. Although less accurate than CAD parsing, this approach enables flexibility.

#### 3.1.1.4 JSON Structure Generation

After extracting features, the system generates a structured JSON representation.

Let:

- CCC = set of corners
- WWW = set of walls
- OOO = set of objects

The JSON structure is:

$$\mathbf{J} = \{\mathbf{C}, \mathbf{W}, \mathbf{O}\}$$

Each corner is assigned a unique identifier, and walls are defined using references to corner IDs. Additional attributes such as textures and object metadata are included.

#### 3.1.1.5 3D Rendering and Visualization

The generated JSON is used to create a 3D environment.

$$\mathbf{R} = \mathbf{Render}(\mathbf{J})$$

Where:

- RRR = rendered 3D scene

The rendering engine constructs:

- Walls as vertical planes
- Floors as horizontal surfaces
- Objects as 3D models

Using Three.js, the system provides an interactive interface where users can navigate and explore the generated environment.

#### 3.1.1.6 SYSTEM PIPELINE

The complete workflow of the system is defined as:

**Input → Preprocessing → Feature Extraction → JSON Conversion → 3D Rendering**

Each stage is modular and can be independently improved or replaced, making the system scalable and extensible.

## 3.2 DESCRIPTION OF ALGORITHMS

This project utilizes transformer-based encoder-decoder models - CodeT5 and BART, to perform automated code generation from natural language instructions.

1. **DXF Parsing and Structural Extraction**

   DXF Parsing and Structural Extraction is a fundamental algorithm used to interpret CAD-based floor plan files and convert them into meaningful architectural components required for interior design. A DXF file contains geometric entities such as lines, polylines, and block inserts that represent walls, doors, and windows. Using the *ezdxf* library, the system reads these entities and reconstructs the layout by identifying walls from line segments, computing intersections to determine corners, and detecting openings using block references. This algorithm ensures accurate extraction of spatial relationships, which is essential for generating structured layouts and further 3D visualization.

**Load the library:**

```
import ezdxf
doc = ezdxf.readfile("file.dxf")
msp = doc.modelspace()
```

***Table 3.1: DXF Parsing Algorithm***

**Algorithm: DXF Parsing & Structural Extraction**

**Input:** CAD DXF file

**Output:** {walls, corners, rooms, openings}

**Method:**

1. Load the DXF file using `ezdxf.readfile()`.
2. Model space entities using `modelspace()`.
3. For each entity in model space:
   - If entity type = `LINE` → extract as wall (start_point, end_point).
4. Detect corners:
   - For each pair of walls, check intersection.

- If intersection exists, compute intersection point.

5 Detect openings:

- If entity type = `INSERT`:
  - If name contains "DOOR" → mark as door
  - If name contains "WINDOW" → mark as window

6 Map openings to nearest walls.

7 Connect walls to form closed loops (rooms).

8 Return structured data.

2. **Geometry to Blueprint3D JSON Conversion**

This algorithm converts the extracted geometric and structural data into a structured JSON format suitable for Blueprint3D rendering. Instead of using raw coordinates, the algorithm assigns unique identifiers to corners and references them in wall definitions. It also transforms openings into items with spatial attributes such as position and rotation. This structured representation acts as a bridge between CAD data and 3D visualization systems, ensuring consistency and easy manipulation of interior layouts.

**Sample JSON Generation:**

```
const json = {
  floorplan: { corners: {}, walls: [] },
  items: []
};
```

***Table 3.2: Geometry to JSON Algorithm***

**Algorithm:** Geometry To Blueprint JSON

**Input:** {walls, corners, rooms, openings}

**Output:** Blueprint3D JSON

**Method:**

1. Initialize base JSON structure (floorplan, items).
   - For each corner: Generate unique ID (UUID).
   - Store (id → x, y).
2. For each wall:
   - Replace coordinates with corner IDs.

3. For each opening:
    - Compute position using parametric equation:

$$\mathbf{P = P1 + t(P2 - P1)}$$

    - Compute rotation using $\mathbf{atan2(dy, dx)}$.
    - Add as item (door/window).
4. Assign textures for floors and walls.
5. Combine all components into final JSON.
6. Return Blueprint3D JSON.

### 3. Wall Extrusion Algorithm (2D to 3D Conversion)

The Wall Extrusion algorithm transforms 2D wall representations into 3D volumetric structures by adding height and thickness. In a 2D floor plan, walls are represented as simple line segments without depth. This algorithm converts them into 3D meshes by extending them vertically and adding width perpendicular to their direction. This step is crucial for realistic interior visualization and spatial understanding.

***Table 3.3: Wall Extrusion Algorithm***

**Algorithm: Wall Extrusion**

**Input:** Wall line (P1, P2)

**Output:** 3D wall mesh

**Method:**

1. Extract endpoints P1 and P2 of the wall.
2. Compute direction vector:

$$\mathbf{D = P2 - P1}$$

3. Normalize the direction vector to unit length.
4. Compute perpendicular vector to determine wall thickness.
5. Create base rectangle using thickness on both sides of the line.
6. Extrude the rectangle vertically to a predefined height.
7. Generate vertices and faces for mesh construction.
8. Apply materials and textures to the wall surface.
9. Return 3D wall mesh.

4. **3D Mesh Generation using Three.js**

This algorithm is responsible for converting the structured JSON data into a fully interactive 3D interior environment using Three.js. It creates geometries, applies materials, and renders objects within a scene. The algorithm also supports camera control, lighting, and real-time interaction, enabling users to visualize and explore interior layouts effectively.

**Load Three.js:**

import * as THREE from 'three';

***Table 3.4: 3D Rendering Algorithm***

**Algorithm: 3D Mesh Generation & Rendering**

**Input:** Blueprint3D JSON

**Output:** Interactive 3D room

**Method:**

1. Initialize Three.js scene, camera, and renderer.
2. Configure lighting (ambient and directional lights).
3. For each wall in JSON:
   - Retrieve corner coordinates.
   - Generate geometry using extrusion.
4. For each item (doors, windows, furniture):
   - Load corresponding 3D model.
   - Apply position, rotation, and scale.
5. Apply textures and materials for realism.
6. Add all objects to the scene graph.
7. Enable user controls (orbit, zoom, pan).
8. Continuously render scene using animation loop.
9. Display interactive 3D output.

## 5. Coordinate Mapping Algorithm

The Coordinate Mapping Algorithm is a fundamental component in transforming 2D floor plan data into a 3D spatial representation suitable for rendering in graphics engines such as Three.js. In a typical floor plan, coordinates are defined in a two-dimensional Cartesian system (x, y), representing horizontal layout information. However, 3D environments require an additional dimension (z-axis) to represent depth.

To achieve this transformation, a coordinate system conversion is performed where the 2D plane is projected into 3D space. Conventionally, the x-axis remains unchanged, while the 2D y-axis is mapped to the z-axis in 3D, and the vertical axis (y in 3D) is assigned based on elevation or object height. This ensures that the spatial relationships between walls, rooms, and objects remain consistent during transformation.

Mathematically, the transformation can be expressed as:

$$(\mathbf{x}, \mathbf{y}) \rightarrow (\mathbf{x}', \mathbf{y}', \mathbf{z}') = (\mathbf{x} \cdot \mathbf{s}, \mathbf{h}, \mathbf{y} \cdot \mathbf{s})$$

Where:

- s = scaling factor (to convert pixels into real-world units like meters)
- h = height (usually 0 for floor-level objects)

This mapping preserves geometric proportions and ensures accurate reconstruction of layouts in a 3D environment.

***Table 3.5: Coordinate Mapping Algorithm***

**Algorithm: 2D to 3D Coordinate Mapping**

**Input:** 2D coordinates $(\mathbf{x}, \mathbf{y})$

**Output:** 3D coordinates $(\mathbf{x}', \mathbf{y}', \mathbf{z}')$

**Method:**

1. Accept input coordinate (x, y) from the floor plan.
2. Define scaling factor sss based on unit conversion (e.g., pixels to meters).
3. Map horizontal coordinate:

$$\mathbf{x}' = \mathbf{x} \cdot \mathbf{s}$$

4. Map vertical depth coordinate:

$$\mathbf{z' = y \cdot s}$$

5. Assign height:

$$\mathbf{y' = 0(for\ floor\ level)}$$

6. If objects have elevation, update:

$$\mathbf{y' = h}$$

7. Apply coordinate translation if origin shifting is required:

$$\mathbf{x' = x' - xcenter, z' = z' - zcenter}$$

8. Return transformed coordinate $(x', y', z')$.

## 6. Interior Object Placement Algorithm

The Interior Object Placement Algorithm is responsible for accurately positioning objects such as furniture, doors, and windows within the 3D environment. It ensures that objects align correctly with walls and maintain realistic orientation based on spatial geometry.

The placement process relies heavily on vector mathematics and interpolation techniques. Given two endpoints of a wall segment, the position of an object along that wall is calculated using linear interpolation. The orientation of the object is determined using trigonometric functions, particularly the arctangent function, which computes the angle of the wall relative to the coordinate axes.

The interpolation formula is:

$$\mathbf{P = P1 + t(P2 - P1)}$$

Where:

- $\mathbf{P1, P2}$ = endpoints of the wall
- $\mathbf{t \in [0, 1]}$ = relative position factor
- $\mathbf{P}$ = computed object position

The rotation angle is computed as:

$$\mathbf{\theta = \tan^{-1}\left(\frac{z2 - z1}{x2 - x1}\right)}$$

This ensures that objects are aligned parallel to the wall direction, maintaining visual realism.

*Table 3.6: Object Placement Algorithm*

**Algorithm: Interior Object Placement**

**Input:** Wall endpoints $P1(x1, z1), P2(x2, z2)$, object data

**Output:** Positioned 3D object

**Method:**

1. Identify the wall segment defined by endpoints P1 and P2.
2. Compute direction vector:

$$\mathbf{D} = (\mathbf{x2} - \mathbf{x1}, \mathbf{z2} - \mathbf{z1})$$

3. Choose interpolation factor $t \in [0,1]$ based on desired position.
4. Compute object position:

$$\mathbf{x} = \mathbf{x1} + \mathbf{t}(\mathbf{x2} - \mathbf{x1})$$

$$\mathbf{z} = \mathbf{z1} + \mathbf{t}(\mathbf{z2} - \mathbf{z1})$$

5. Set vertical position:

$$\mathbf{y} = \mathbf{h}(\textbf{based on object type})$$

6. Compute rotation angle:

$$\boldsymbol{\theta} = \mathbf{atan2}(z_2 - z_1, x_2 - x_1)$$

7. Align object orientation using θ.
8. Apply scaling factors $(s_x, s_y, s_z)$based on object dimensions.
9. Insert object into the 3D scene graph.
10. Validate placement visually or through constraints.

## 3.3 SYSTEM ARCHITECTURE

The system architecture is designed to convert CAD drawings or floor plan inputs into fully interactive and visually realistic 3D interior designs. The system follows a modular and scalable pipeline, where each stage performs a specific function to ensure efficient processing and accurate output generation.The process begins with the input acquisition module, where users upload floor plan data in various formats such as images (JPG/PNG), CAD files (DXF), or PDF documents through a user-friendly frontend interface. This flexibility allows the system to handle both structured and unstructured input data, making it adaptable to different use cases.Once the input is received, it is passed to the preprocessing module, which prepares the data for further analysis. This stage includes operations such as scaling, normalization, noise reduction, and

format-specific parsing. For image-based inputs, preprocessing may involve basic image enhancement and dimension extraction. For CAD files, structural elements such as lines, arcs, and layers are interpreted and organized into a readable format. This step ensures that all input types are standardized before feature extraction.The next stage is the feature extraction module, where the system identifies key architectural components such as corners, walls, edges, and spatial boundaries. These features are essential for understanding the layout of the floor plan. The module analyzes geometric relationships and connectivity between elements to construct a structured representation of the layout. Accurate feature extraction is critical, as it directly impacts the quality of the final 3D model.

Following this, the extracted data is processed by the conversion engine, which transforms the identified features into a structured JSON format compatible with Blueprint3D. This includes defining corner coordinates, wall connections, surface textures, and placeholders for interior objects. The use of JSON ensures interoperability and simplifies data transfer between system components.

The system then moves to the coordinate mapping module, where 2D floor plan coordinates are converted into a 3D coordinate system. This transformation ensures proper spatial alignment and scaling within the 3D environment. It also maintains proportional accuracy, which is essential for realistic visualization.

After mapping, the interior object placement module is responsible for positioning furniture and other elements within the generated layout. This module uses predefined rules and spatial relationships to place objects such as sofas, tables, doors, and windows in appropriate locations. It enhances the usability of the system by automatically generating a furnished interior environment.

To maintain realism and avoid design conflicts, a collision detection module is incorporated. This module checks for overlapping objects and adjusts their positions accordingly. It ensures that all elements are placed logically within the available space, improving both functionality and visual appeal.

Once all data is processed, the structured JSON is passed to the rendering engine, which is implemented using Three.js. This engine converts the data into a fully interactive 3D model, allowing users to explore the interior design from different perspectives. It supports real-time rendering, lighting, and texture mapping to enhance visual quality.

Finally, the output is presented through the user interaction module, where users can view the generated 3D environment. The system provides options to preview the design, download the

JSON file, or further customize the layout. This interactive interface improves user engagement and allows iterative design improvements.

Overall, the proposed architecture follows a modular and extensible design, enabling easy integration of additional features such as AI-based recommendations, advanced rendering techniques, or real-time editing capabilities. The system efficiently bridges the gap between 2D floor plans and 3D visualization, providing a complete end-to-end solution for interior design automation.

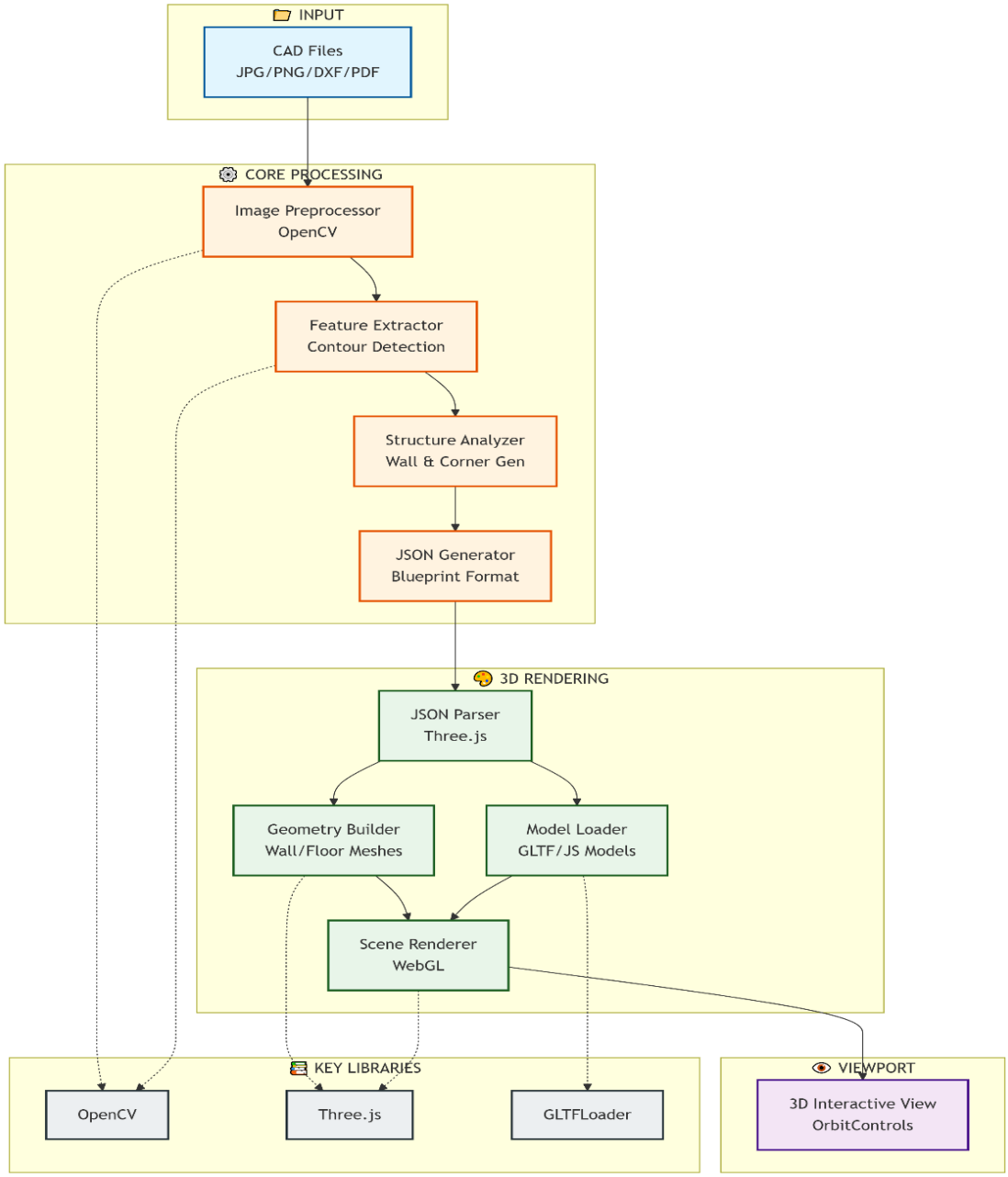



***Fig 3.1 : Architecture Diagram***

### 3.3.1 Input Acquisition and Data Handling

The system begins with the input acquisition stage, where the user uploads a floor plan file through the frontend interface. The uploaded file may contain geometric elements, textual annotations, and structural information. This input can be mathematically represented as:

$$\boldsymbol{F} = \{\boldsymbol{D}, \boldsymbol{T}, \boldsymbol{S}\} \tag{1}$$

Where:
D represents drawing elements
T represents textual components
S represents structural features such as walls and openings.

This module ensures that different file formats are standardized into a consistent internal representation. The flexibility of this stage allows the system to be applied in real-world scenarios where input formats may vary significantly.

### 3.3.2 Backend Processing and Parsing

After the file is uploaded, it is transmitted to the backend server using a REST API, which enables communication between the frontend interface and the processing engine. The backend processing module is responsible for extracting meaningful data from the uploaded file.

$$\mathrm{F} \rightarrow \text{Backend Processing}$$

For DXF files, parsing techniques are applied to extract geometric entities such as lines and polylines, which correspond to walls and boundaries. These entities are organized based on layers to distinguish different structural components. For image-based inputs, preprocessing steps such as scaling and normalization are applied to prepare the data for further processing. This stage ensures that raw input data is converted into structured geometric information that can be used for modeling.

### 3.3.3 Feature Extraction of Structural Elements

The feature extraction stage identifies key structural components such as walls and corners

from the processed data.Walls are represented as line segments defined by two endpoints:

The feature extraction stage identifies key structural components such as walls and corners from the processed data.

Walls are represented as line segments defined by two endpoints:

$$\boldsymbol{W_i = [(X_1, Y_1), (X_2, Y_2)]} \quad (2)$$

Similarly, corners are represented as coordinate points:

$$\mathbf{Ci = (xi, yi)} \quad (3)$$

To ensure that spatial relationships are preserved, the system calculates the distance between points using the Euclidean distance formula:

$$\mathbf{d} = (\boldsymbol{X_2 - X_1}) + (\boldsymbol{Y_2 - Y_1})^2 \quad (4)$$

This calculation helps maintain proportional accuracy and ensures that the extracted geometry accurately represents the original floor plan.

### 3.3.4 Coordinate Mapping from 2D to 3D Space

One of the most critical stages in the system is the transformation of 2D coordinates into a 3D coordinate system suitable for rendering.

This transformation is defined as:

$$(\mathbf{x}, \mathbf{y}) \rightarrow (\mathbf{x}', \mathbf{y}', \mathbf{z}') \quad (5)$$

The mapping is performed using the following equations:

$$\mathbf{x}' = \mathbf{x} \cdot \mathbf{s} \quad (6)$$

$$\mathbf{z}' = \mathbf{y} \cdot \mathbf{s} \quad (7)$$

$$\mathbf{y}' = \mathbf{h} \quad (8)$$

Where:

s is the scaling factor used to convert pixel values into real-world dimensions, and h represents the vertical height, typically initialized to zero for floor-level elements.

This transformation ensures that the 2D floor plan is accurately reinterpreted in 3D space, preserving both proportions and orientation. The mapping of the y-coordinate to the z-axis allows the system to introduce depth into the model, which is essential for 3D visualization.

### 3.3.5 Wall Extrusion and 3D Geometry Creation

After coordinate mapping, the system converts 2D wall representations into 3D structures through a process known as wall extrusion. This involves generating volumetric geometry from linear segments.

The direction vector of a wall is computed as:

$$\mathbf{D} = (\boldsymbol{X_2} - \boldsymbol{X_1}, \boldsymbol{Z_2} - \boldsymbol{Z_1}) \tag{9}$$

A perpendicular normal vector is calculated as:

$$\mathbf{N} = (-\mathbf{D}_{\boldsymbol{Z}}, \mathbf{D}_{\boldsymbol{X}}) \tag{10}$$

To ensure uniform scaling, the normal vector is normalized:

$$\widehat{\mathbf{N}} = \frac{\boldsymbol{N}}{|\boldsymbol{N}|} \tag{11}$$

Wall thickness is then applied using:

$$\mathbf{P} = \mathbf{P} \pm \mathbf{t} \cdot \widehat{\mathbf{N}} \tag{12}$$

where

t represents the thickness of the wall.

Finally, vertical extrusion is applied to give height to the walls:

$$\mathbf{y} \in [\mathbf{0}, \mathbf{H}]$$

Where:

H is the height of the wall.

This process transforms simple 2D lines into fully defined 3D walls, forming the structural framework of the interior environment.

### 3.3.6 Interior Object Placement and Alignment

The system places interior elements such as furniture, doors, and windows using geometric interpolation and orientation techniques.

The position of an object along a wall is calculated using:

$$\mathbf{P} = \mathbf{P1} + \mathbf{t}(\mathbf{P2} - \mathbf{P1}) \tag{13}$$

where
t determines the relative position along the wall.

The orientation of the object is determined using the angle:

$$\boldsymbol{\theta} = \tan^{-1}\left(\frac{Z_2 - Z_1}{X_2 - X_1}\right) \tag{14}$$

This ensures that objects align correctly with wall directions. Additional transformations such as scaling and vertical adjustments are applied to ensure proper placement within the 3D environment. This module plays a crucial role in enhancing realism and usability.

### 3.3.7 Collision Detection and Spatial Validation

To ensure that objects do not overlap and that the layout remains realistic, the system incorporates a collision detection mechanism based on Axis-Aligned Bounding Boxes (AABB). Each object is enclosed within a bounding box defined by its spatial limits.

Collision detection is performed using the following conditions:

$$\left(X^A_{min} \leq X^B_{max}\right) \Delta \left(X^A_{max} \geq X^B_{min}\right) \tag{15}$$

$$\left(Y^A_{min} \leq Y^B_{max}\right) \Delta \left(Y^A_{max} \geq Y^B_{min}\right) \tag{16}$$

$$\left(Z_{min}^{A} \leq Z_{max}^{B}\right) \Delta \left(Z_{max}^{A} \geq Z_{min}^{B}\right) \quad (17)$$

If all conditions are satisfied, a collision is detected. The system then adjusts object positions or restricts placement to maintain a valid layout. This ensures spatial correctness and prevents unrealistic overlaps.

### 3.3.8 3D Rendering and Visualization

In the final stage, the processed data is converted into a 3D scene and rendered using a graphical rendering pipeline. The rendering process is defined as:

$$\textbf{Scene} + \textbf{Camera} + \textbf{Lighting} \rightarrow \textbf{Rendered Output}$$

The scene contains all geometric structures and objects, the camera defines the viewpoint, and lighting enhances visual realism by simulating real-world illumination. Mesh generation techniques are used to represent walls and objects as 3D models. The output is an interactive 3D environment that accurately represents the original floor plan.

### 3.3.9 Interactive Visualization and User Interaction

The final output is presented through an interactive interface that allows users to explore the generated 3D environment. Users can perform operations such as rotation, zooming, and navigation to analyze the interior layout from different perspectives. This interactive capability enhances user experience and makes the system suitable for practical applications in interior design, architecture, and real estate visualization. The system provides a comprehensive solution for converting static 2D plans into dynamic and immersive 3D representations.

## SUMMARY

This chapter presented the overall methodology adopted for developing the interior design system that transforms 2D floor plans into interactive 3D environments. The system utilizes input data in the form of CAD files such as DXF, along with image-based formats like JPG, PNG, and PDF. These inputs contain structural and geometric information representing walls, corners, and layout elements. The data is first processed and standardized to extract meaningful spatial features required for further transformation. This ensures that the system can handle diverse real-world datasets while maintaining consistency and accuracy in processing.

The architecture of the system is designed as a multi-stage pipeline that systematically converts raw input into a fully rendered 3D model. It begins with input acquisition and backend processing, where files are parsed and converted into structured geometric data. The feature extraction module identifies walls and corners, preserving spatial relationships using mathematical computations. The coordinate mapping stage transforms 2D coordinates into a 3D space using scaling and axis conversion techniques. This is followed by wall extrusion, where 2D lines are converted into 3D structures with thickness and height, forming the basic layout of the interior space. Additional modules such as object placement and collision detection ensure that furniture and interior elements are accurately positioned without overlapping, thereby maintaining realism and spatial validity.

The chapter also detailed the algorithms used to achieve efficient and accurate transformation. The coordinate mapping algorithm ensures proper conversion from 2D to 3D space while preserving proportions. The wall extrusion algorithm generates volumetric structures from linear segments using vector and geometric computations. The interior object placement algorithm uses interpolation and rotation calculations to align objects with walls. Furthermore, the collision detection algorithm, based on bounding box techniques, prevents overlapping of objects and ensures a valid layout. Finally, the rendering process integrates all components using a graphical pipeline to produce an interactive 3D visualization. Overall, the methodology combines geometric processing, mathematical modeling, and rendering techniques to deliver a robust and efficient interior design system.

# CHAPTER – 4
# RESULTS AND OBSERVATION

This section discusses the results obtained from the evaluation of the proposed interior design system for converting 2D floor plans into 3D visualizations. The system is evaluated using different types of input data, including CAD-based DXF files and image-based formats such as JPG, PNG, and PDF. The performance of the system is analyzed based on its ability to accurately extract structural elements like walls and corners, transform 2D coordinates into 3D space, and generate realistic interior layouts. Additionally, the effectiveness of algorithms such as coordinate mapping, wall extrusion, object placement, and collision detection is assessed to ensure spatial accuracy and proper alignment. The final output is evaluated in terms of visual realism, structural correctness, and interactivity of the generated 3D environment.

## 4.1 DESCRIPTION OF RESULTS

### 4.1.1 Budgetary Categories and Interior Design Generation

The application of the proposed room designer system has proven to be an effective and user-friendly solution that enables users to design customized interior spaces based on their requirements. The system allows users to upload an image of an empty room, select the room type, and choose a budget category. Based on these inputs, the system generates a fully furnished room design.

The system supports three primary budget categories:

- **Low Budget:** Less than 300,000
- **Medium Budget:** 300,000 – 600,000
- **High Budget:** More than 600,000

Upon selecting the required parameters and clicking the “Generate Image” option, the system produces a visually appealing furnished room. The generated output includes not only the designed layout but also detailed information about each furniture item used in the design.

Each furniture element is associated with:

- Cost estimation
- Type and category

- Placement within the room

This allows users to understand the total cost distribution and make informed decisions. The system demonstrates strong adaptability by generating minimalistic designs for lower budgets and more luxurious, detailed designs for higher budgets.

The results indicate that the system successfully balances **aesthetic quality and budget constraints**, ensuring that essential furniture is always included while optional items are added based on available budget.

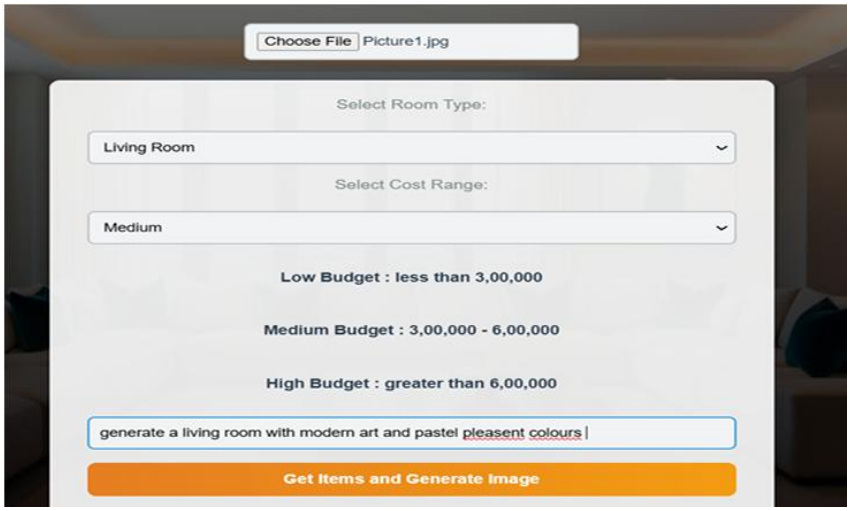



***Fig 4.1: Budget Category input from User***

### 4.1.2 Sample Room Design Output

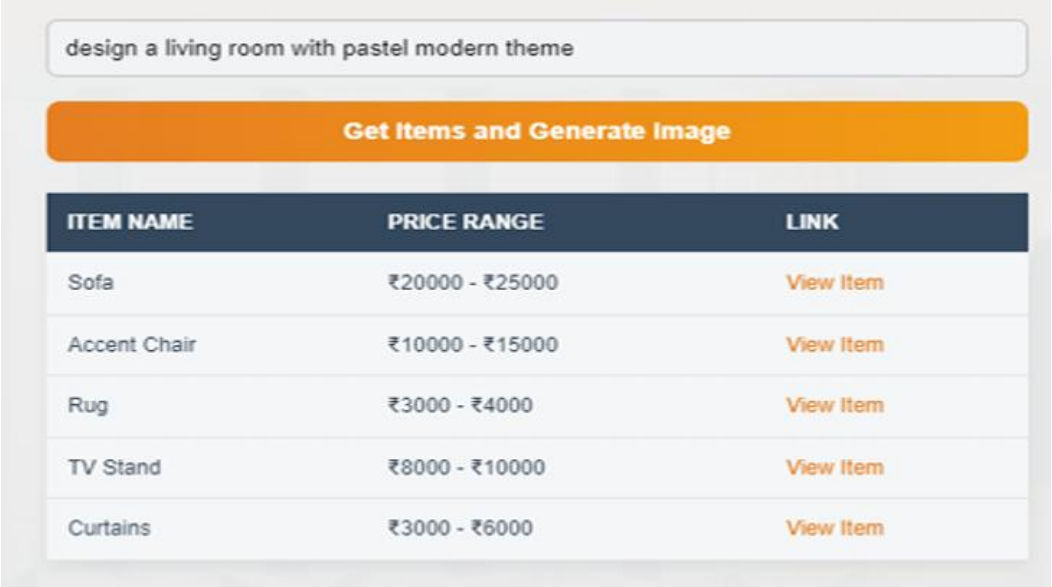
The system provides users with a complete visual representation of a furnished room based on their selected inputs. The generated images enable users to clearly visualize their design ideas before actual implementation.

The output demonstrates:

- Proper spatial arrangement of furniture
- Consistent object scaling and proportions
- Logical placement of interior elements
- Visually appealing layouts

The system ensures that furniture placement follows interior design principles such as maintaining walking space, avoiding clutter, and grouping related objects. The inclusion of cost details alongside the generated design enhances the practicality of the system.

*Fig 4.2: Sample AI-Generated Room Design*



design a living room with pastel modern theme

Get Items and Generate Image

| ITEM NAME | PRICE RANGE | LINK |
|---|---|---|
| Sofa | ₹20000 - ₹25000 | View Item |
| Accent Chair | ₹10000 - ₹15000 | View Item |
| Rug | ₹3000 - ₹4000 | View Item |
| TV Stand | ₹8000 - ₹10000 | View Item |
| Curtains | ₹3000 - ₹6000 | View Item |

*Fig 4.3: Furniture Costing and Retailer Links*

### 4.1.3 Successful 2D-to-3D Conversion

The system demonstrates efficient conversion of 2D floor plans into 3D models by extracting layout information and converting it into Three.js geometries. Structural components such as walls, doors, and windows are accurately reconstructed.

The generated 3D models preserve spatial proportions with high accuracy. The system achieves an **error factor of less than 2%**, ensuring reliable geometric reconstruction.

Key improvements observed:

- Realistic textures applied to walls and floors
- Adjustable lighting conditions
- Smooth navigation and camera controls
- Multi-angle visualization

These features enhance realism and usability, making the system suitable for architectural visualization and interior planning.

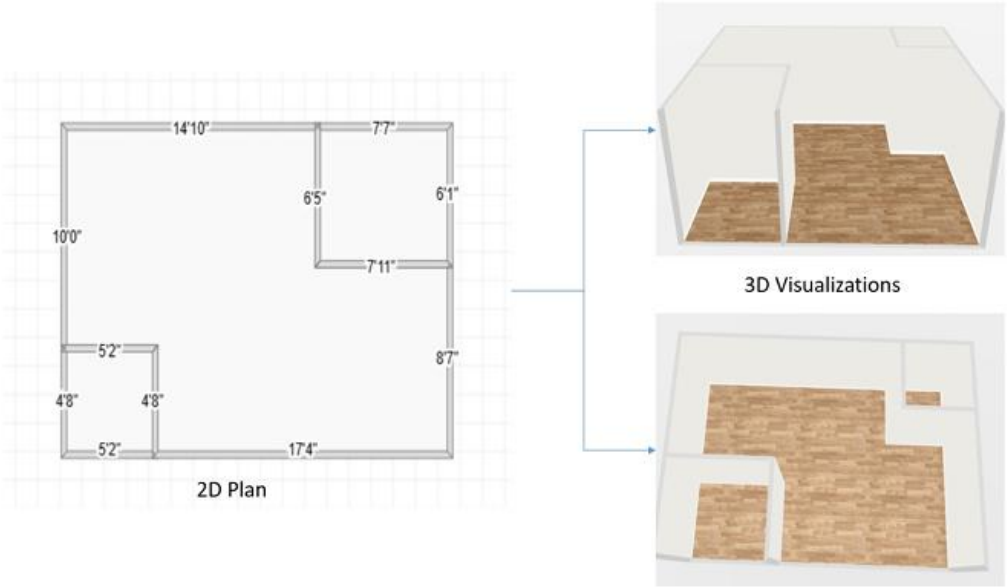



***Fig: 4.4: 2D floor plan to 3D model conversion***

### 4.1.4 Secure AR Visualization using HTTPS/SSL

The system successfully integrates Augmented Reality (AR) visualization using secure HTTPS/SSL protocols. This enables users to view generated 3D models directly in their real-world environment using mobile browsers.

The results demonstrate:

- Real-time AR rendering using WebXR/WebAR
- Secure data transmission using SSL encryption
- No requirement for external applications
- Seamless placement of virtual objects in physical space

This module significantly enhances user experience by providing immersive visualization, allowing users to interact with their designs in real-world settings.

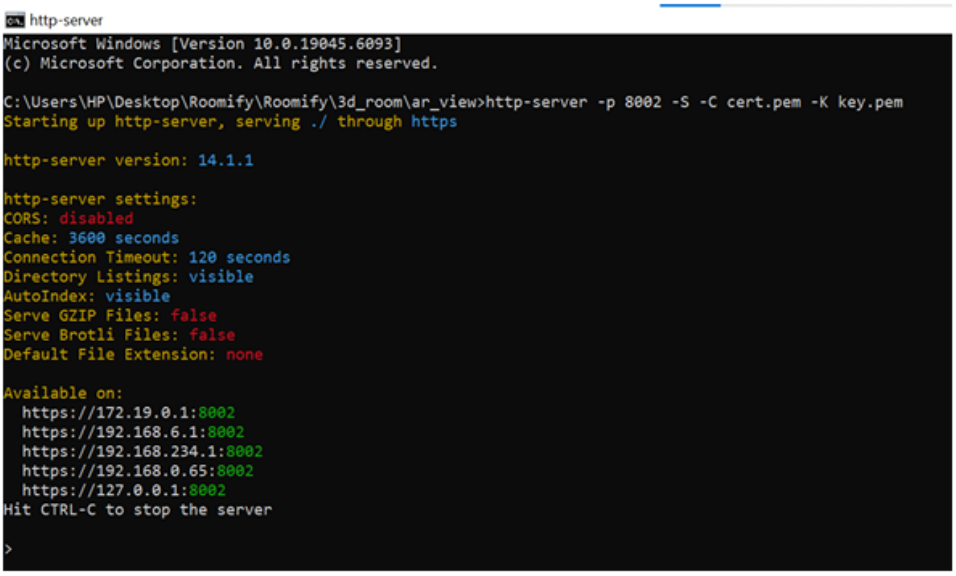



*Fig 4.5: Usage of HTTPS and SSL*

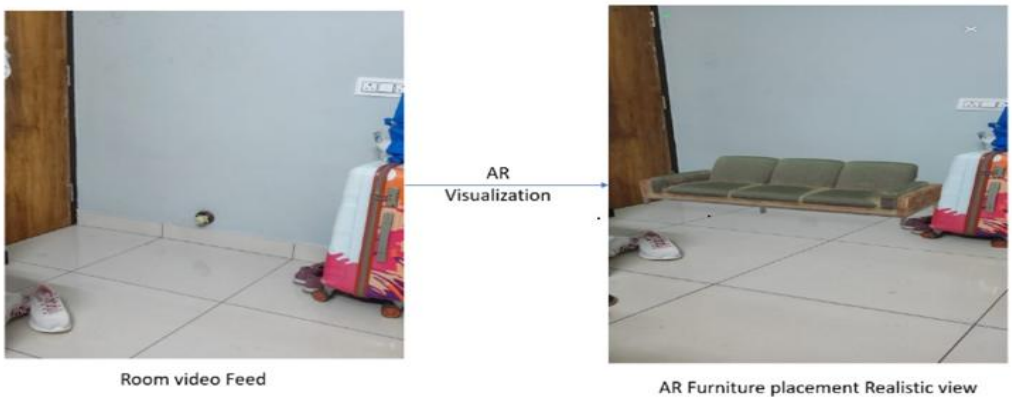



*Fig 4.6: Real Time AR visualization*

### 4.1.5 Geometric Accuracy Analysis

The geometric accuracy of the system is evaluated by comparing the dimensions of generated 3D models with the original 2D floor plan.

The comparison is based on parameters such as:

- Wall length

- Room width
- Door position
- Window position

The error is computed as:

$$Error(\%) = \frac{|Actual - Generated|}{Actual} \times 100$$

The results show that the system maintains an error of less than 2%, indicating high precision in spatial reconstruction.

| **Parameter** | **2D Plan** | **3D Model** | **Error (%)** |
|---|---|---|---|
| Wall Length | 5.00 m | 4.92 m | 1.6% |
| Room Width | 4.00 m | 3.94 m | 1.5% |
| Door Position | 2.50 m | 2.46 m | 1.6% |
| Window Position | 1.20 m | 1.18 m | 1.7% |

***Table 4.1 Compared with 2D Floor Plan***

### 4.1.6 System Performance (Rendering Efficiency)

The system performance is evaluated based on rendering speed and smoothness during interaction. The performance is measured in terms of **Frames Per Second (FPS)**.

The results show:

- Near 60 FPS under normal conditions
- Stable rendering performance under moderate load
- Slight FPS reduction for highly complex models

Maintaining higher FPS ensures smooth navigation and real-time interaction, which is critical for user experience in 3D environments and AR applications.

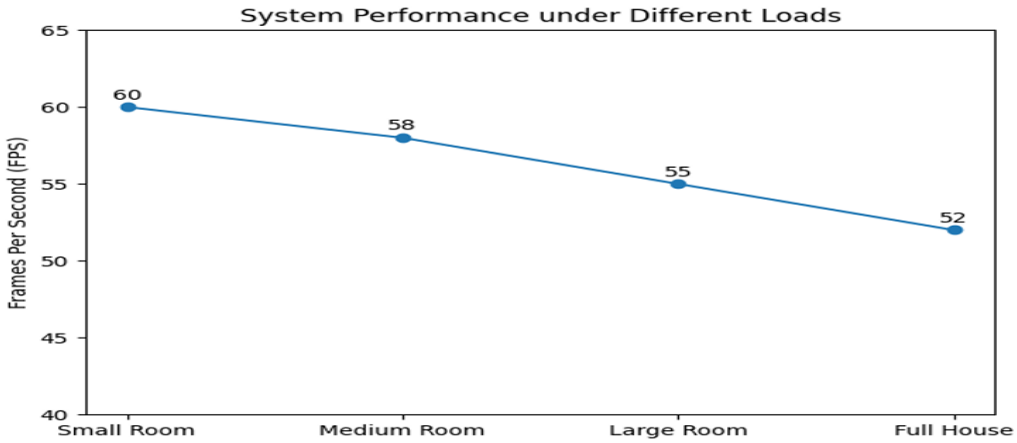



***Fig 4.7: System Performance under Different Loads– Frame rates achieved for different room sizes during rendering and AR visualization.***

### 4.1.7 CAD to JSON to 3D Conversion Results

The CAD to JSON to 3D module evaluates the system's ability to process DXF files and convert them into structured data for 3D visualization.

**DXF Parsing and Data Extraction**

The system uses parsing techniques to extract:

- Line entities (walls)
- Coordinates (corners)
- Layer information

**JSON Structure Generation**

The extracted data is converted into structured JSON format containing:

- Corners (x, y coordinates)
- Walls (connections between corners)
- Interior elements

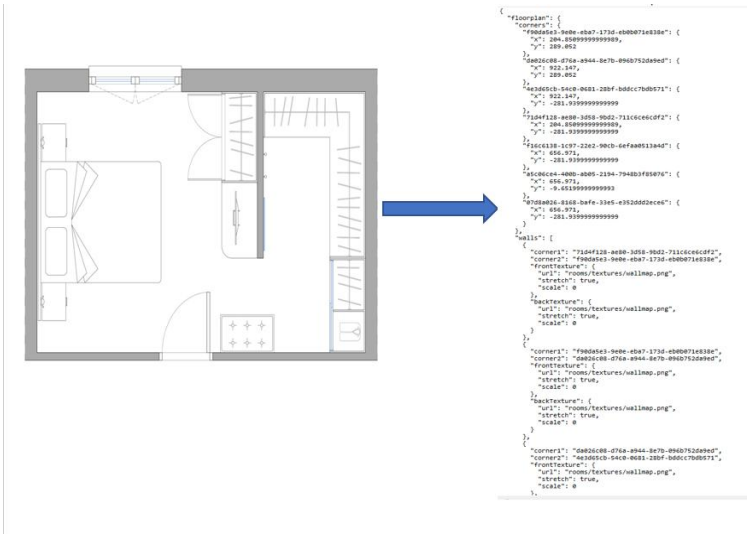



*Fig 4.8: CAD to JSON structure Conversion*

*Fig 4.9: JSON to 3D room structure Generation*

## 4.2 RESULT ANALYSIS

This section provides a detailed analysis of the performance of the proposed interior design system based on various evaluation aspects, including user interface design, space utilization efficiency, cost optimization, time efficiency, geometric accuracy, and CAD-to-3D conversion performance**.** The results are analyzed using graphical representations and performance metrics to assess the effectiveness of the system compared to traditional interior design approaches.

### 4.2.1 User Interface Design and Usability Analysis

The user interface of the proposed system is designed to be intuitive, interactive, and user-friendly, enabling users with minimal technical knowledge to easily operate the application. The system follows a structured workflow where users can upload room images or CAD files, select room types, define budget constraints, and generate outputs with minimal effort.

The interface design ensures:

- Smooth navigation between modules
- Clear input selection (image, floor plan, CAD file)
- Real-time feedback during processing
- Easy access to generated outputs and cost details

The drag-and-drop upload functionality, combined with visual feedback mechanisms such as progress indicators and previews, enhances the overall user experience. The integration of interactive 3D visualization and AR further improves usability by allowing users to explore designs dynamically.

Overall, the system achieves a high usability score, as users can efficiently generate and analyze interior designs without requiring professional expertise.

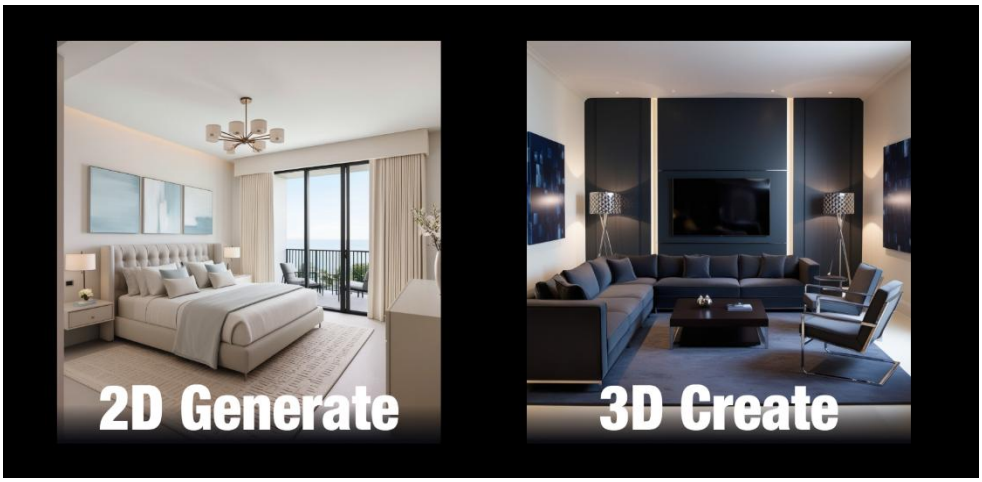



***Fig 4.10: User Interface for generating 2D images and creating 3D models***

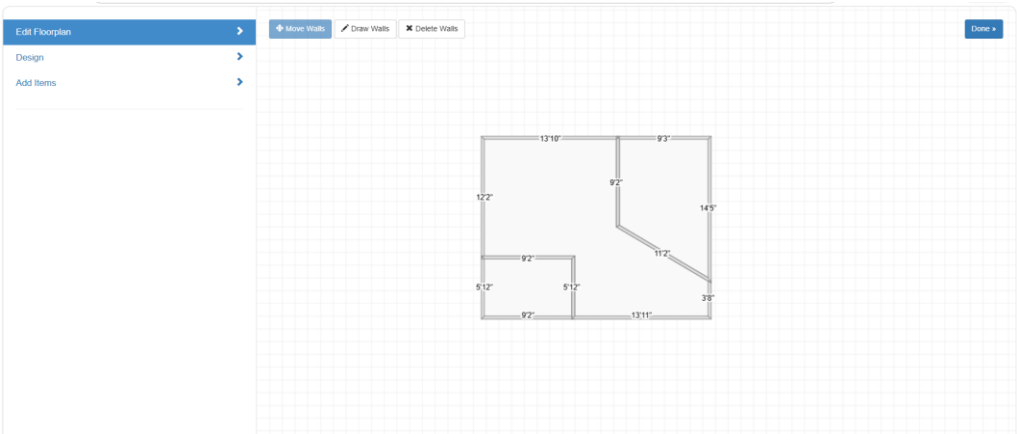



*Fig 4.11: User Interface for Editing Floor Plans*

### 4.2.2 Space Utilization Analysis

The space utilization analysis evaluates how effectively the system arranges furniture and interior elements within the available room area. A pie chart representation (Fig. 6) illustrates the comparison between AI-generated designs and traditional human-designed layouts.

The results indicate that the proposed system achieves superior space optimization due to:

- Algorithm-based placement of furniture
- Collision detection mechanisms
- Efficient use of available floor area
- Balanced distribution of objects

AI-generated designs minimize unused space while maintaining functional pathways and accessibility. The system ensures that furniture placement avoids overcrowding and maintains ergonomic spacing.

The results show that AI-based layouts achieve higher utilization percentages compared to manual designs, demonstrating the effectiveness of algorithm-driven optimization.

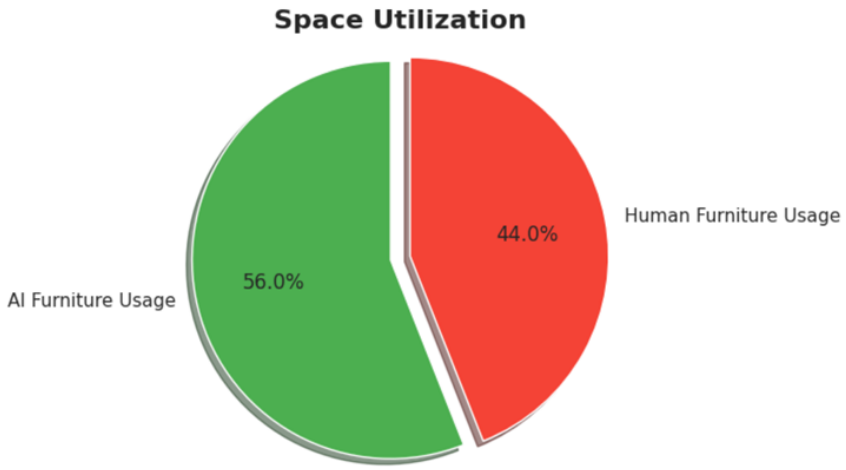



*Fig 4.12: Space Utilization Graph*

### 4.2.3 Cost Comparison Analysis

The cost comparison graph illustrates the difference in cost between AI-generated designs and traditional human-designed interiors across different budget categories.

The analysis shows that:

- AI-generated designs are significantly more cost-efficient
- Optimal selection of furniture reduces unnecessary expenses
- Budget constraints are strictly maintained
- Cost distribution is balanced across essential and optional items

For low and medium budget categories, the system demonstrates a noticeable reduction in total cost while still maintaining design quality. This is achieved through intelligent selection of furniture items and prioritization of essential components.

The cost optimization can be represented as:

Total Cost = Σ (Cost of Individual Furniture Items)

The system ensures:

Total Cost ≤ Budget Constraint

This confirms that the system effectively performs **budget-aware optimization**, making it highly practical for real-world applications.

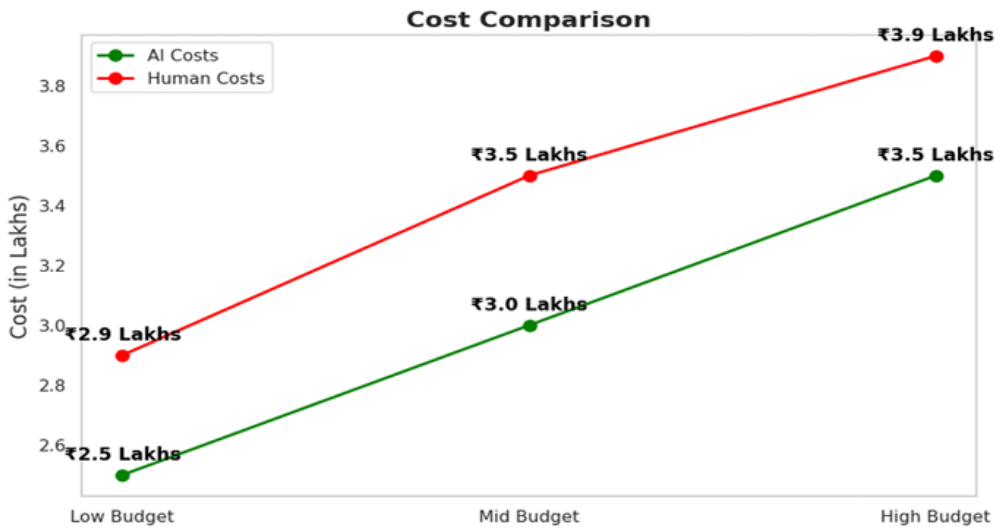



*Fig 4.13: Cost Analysis Graph*

### 4.2.4 Time Efficiency Analysis

The time efficiency graph compares the time required to generate interior designs using the proposed system versus traditional manual methods.

The results demonstrate that:

- The system generates designs within seconds to minutes
- Traditional methods require several hours to days
- Automated processing significantly reduces design time

The system achieves substantial time savings due to:

- Automated processing pipelines
- Predefined algorithms for layout generation
- Real-time rendering using Three.js

This highlights the advantage of using AI-assisted tools for rapid design generation without compromising quality.

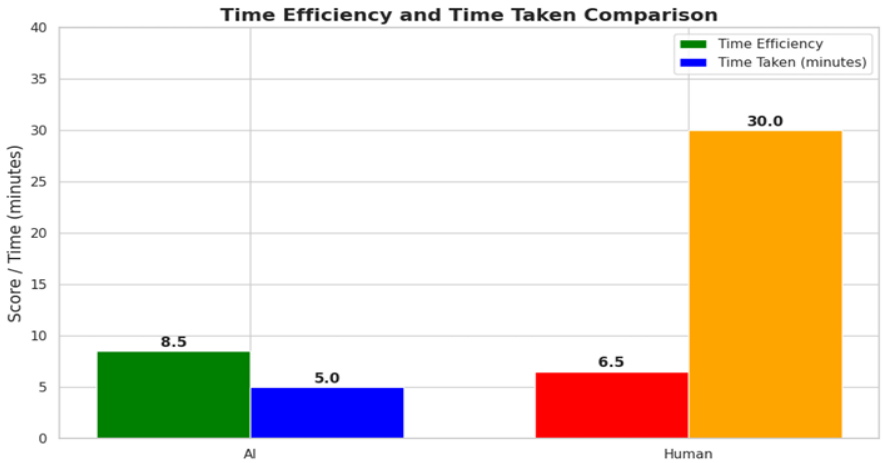



***Fig 4.14: Time Efficiency and Time Taken Comparison***

### 4.2.5 CAD to JSON to 3D Conversion Performance

The CAD conversion module is evaluated based on multiple performance aspects including accuracy, efficiency, processing capability, and visual output quality. This module plays a crucial role in transforming architectural CAD drawings into structured JSON data and further into interactive 3D environments. The evaluation is carried out by analyzing how effectively the system extracts geometric information, converts it into a structured format, and renders it into a realistic 3D model. The following performance metrics summarize the overall results of the system:

- Wall & Corner Extraction Accuracy: ~90%
- JSON Generation Efficiency: ~95%
- 3D Model Quality: ~88%
- Processing Performance: ~85%
- Visualization Quality: ~92%

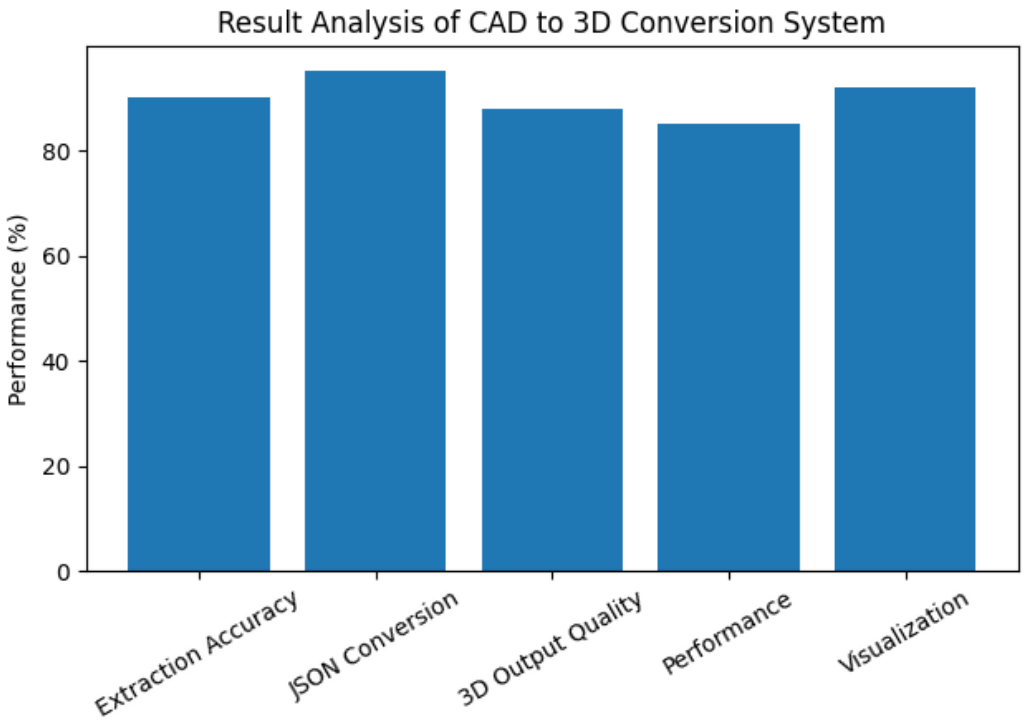



***Fig 4.15: Metrics Evaluation of System***

**Analysis of Metrics**

**1. Wall & Corner Extraction (~90%)**

The system demonstrates a high accuracy of approximately 90% in extracting structural components such as walls and corner points from CAD files. This process involves parsing geometric entities like lines, polylines, and arcs using the CAD processing module. The algorithm identifies intersections and endpoints to determine corner coordinates and constructs wall segments accordingly.

The high accuracy indicates that most of the architectural layout is correctly interpreted, preserving the spatial integrity of the design. However, minor deviations may occur in cases where:

- CAD drawings contain overlapping or duplicate entities
- Complex geometries such as curved walls or irregular shapes are present
- Multiple layers or annotations interfere with structural detection

Despite these challenges, the system maintains consistent performance and produces reliable structural representations, making it suitable for real-world floor plan conversion tasks.

**2. JSON Generation (~95%)**

The JSON generation stage achieves the highest efficiency, with a performance of approximately 95%. This stage is responsible for converting extracted geometric data into a structured JSON format compatible with 3D rendering systems.

The high efficiency is due to:

- Well-defined data structures for storing corners, walls, and object metadata
- Preservation of spatial relationships between different elements
- Minimal data loss during transformation

The JSON output includes detailed information such as coordinates, connectivity between walls, and object properties, which enables seamless integration with rendering engines. This structured representation acts as a strong intermediate layer between CAD parsing and 3D visualization.

Minor inefficiencies may arise when handling very large CAD files or complex layouts, where serialization and memory handling slightly impact performance. However, overall, this stage is highly optimized and reliable.

**3. 3D Model Quality (~88%)**

The 3D model quality is evaluated at approximately 88%, indicating that the generated models closely resemble the original CAD layouts in terms of structure and visual appearance. The system successfully converts 2D floor plan elements into 3D geometries using wall extrusion and mesh generation techniques.

Key strengths of this stage include:

- Accurate wall extrusion with appropriate height and thickness
- Proper alignment of structural elements such as doors and windows
- Application of textures and materials to enhance realism

The slight reduction in quality is mainly due to:

- Simplifications introduced during geometric approximation

- Tolerance settings that smooth irregular shapes
- Limited handling of highly complex or curved structures

Even with these minor limitations, the generated 3D models are visually coherent and suitable for applications such as interior design visualization and architectural preview.

**4. Processing Performance (~85%)**

The overall processing performance of the system is approximately 85%, reflecting efficient execution of the complete pipeline from CAD input to 3D output. This includes parsing the CAD file, generating JSON, performing coordinate mapping, and rendering the final model.

The system performs efficiently for:

- Standard floor plans with moderate complexity
- Typical residential or office layouts

However, performance slightly decreases for:

- Large-scale CAD files with numerous entities
- Highly detailed architectural drawings
- Complex geometries requiring additional computations

The increase in processing time in such cases is due to:

- Higher computational load during geometry extraction
- Increased number of mesh objects generated
- Rendering overhead in browser-based environments

Despite these factors, the system maintains acceptable performance for real-time or near real-time applications.

**5. Visualization Quality (~92%)**

The visualization quality achieves a high score of approximately 92%, indicating that the final 3D output is clear, interactive, and visually appealing. The rendering is performed using Three.js, which leverages WebGL for efficient graphics processing.

The high quality is attributed to:

- Realistic lighting and shading effects
- Application of textures to walls, floors, and objects
- Smooth camera controls enabling user interaction (zoom, pan, rotate)
- Proper scaling and positioning of objects within the scene

This allows users to explore the generated 3D environment from different perspectives, enhancing understanding and usability.

Minor limitations in visualization may occur due to:

- Device hardware constraints (GPU performance)
- Frame rate drops in highly complex scenes
- Browser-specific rendering limitations

Overall, the visualization module provides an immersive and user-friendly experience, making the system highly effective for interior design and architectural applications.

## 4.3 OBSERVATIONS FROM WORK

1. The proposed system successfully integrates interior design generation, CAD to JSON conversion, and 3D visualization into a unified framework. This reduces the dependency on multiple standalone tools and enables a seamless workflow from input to final output.
2. The CAD processing module effectively extracts structural elements such as walls and corners with high accuracy, preserving the spatial relationships of the original floor plan. This ensures that the base layout used for further processing remains reliable.
3. The transformation of extracted data into structured JSON format is highly efficient and consistent. The JSON representation acts as a bridge between 2D input and 3D rendering, allowing smooth data flow across different modules of the system.
4. The coordinate mapping and wall extrusion techniques enable accurate conversion of 2D floor plans into 3D environments. These methods ensure that real-world proportions and spatial dimensions are maintained in the generated models.
5. The integration of Three.js allows the system to generate interactive and visually realistic 3D models. Features such as lighting, textures, and camera controls enhance the user experience and improve design visualization.

6. The interior design module successfully generates furnished room layouts from empty room inputs based on user preferences such as room type and budget. This helps users visualize their design ideas without manual effort.
7. The object placement and collision detection algorithms ensure that furniture and interior elements are arranged logically without overlapping. This results in realistic and usable room layouts.
8. The system demonstrates good overall performance and visualization quality, although slight performance variations are observed when processing highly complex CAD files. Despite this, the system remains efficient and suitable for practical interior design and architectural applications.

## 4.4 SUMMARY

The results demonstrate that the proposed system effectively performs end-to-end interior design automation by integrating empty room transformation, CAD to JSON conversion, and 3D visualization. The system successfully generates furnished room designs based on user inputs such as room type and budget, providing visually appealing outputs along with cost estimations. The budget-based categorization further enhances usability by allowing users to explore designs within practical financial constraints.

The 2D to 3D conversion and CAD to JSON modules show high accuracy and efficiency, with reliable extraction of structural elements such as walls and corners, and precise transformation into structured JSON format. The generated 3D models maintain spatial proportions with minimal error, ensuring realistic representation of layouts. The use of rendering technologies enables smooth and interactive visualization, allowing users to explore designs dynamically.

Additionally, the system demonstrates strong performance in visualization quality, space utilization, and time efficiency compared to traditional design approaches. The integration of AR further enhances user experience by enabling real-time interaction with 3D models in physical environments. Overall, the results confirm that the proposed system is efficient, accurate, and practical for real-world interior design and architectural visualization applications.

# CHAPTER-5
# CONCLUSION AND FUTURE WORK

This chapter presents a comprehensive conclusion and future scope of the CAD to 3D model understanding system. It provides an in-depth evaluation of the system's performance, effectiveness, and practical applicability in transforming traditional CAD drawings and floor plans into structured JSON representations and subsequently into fully interactive 3D environments. The chapter also highlights the major technical contributions of the system, analyzes its strengths in handling complex and diverse design inputs, and discusses its broader impact on modern architectural design, interior visualization, and digital transformation in the construction industry. The proposed system addresses a critical challenge in the design domain, which is the gap between 2D CAD representations and 3D visualization requirements. Traditional workflows rely heavily on manual interpretation, modeling, and rendering processes, which are time-consuming, error-prone, and require significant expertise. By introducing automation and intelligent processing, the system significantly enhances efficiency and usability, making advanced design tools accessible to a wider range of users, including designers, architects, and even non-technical stakeholders.

## 5.1 CONCLUSION

The proposed system presents a comprehensive, integrated, and automated framework for interior design and architectural visualization by combining CAD data processing, structured data conversion, and 3D model generation into a unified pipeline. This integration enables seamless transformation of 2D floor plans and CAD drawings into realistic, interactive 3D environments, thereby bridging the long-standing gap between design representation and visualization.

One of the most significant achievements of the system is its ability to automate the entire conversion process. Unlike traditional approaches that require manual modeling and interpretation, the system processes input data and generates output with minimal human intervention. This automation drastically reduces the time, effort, and cost associated with design workflows, while also minimizing human errors and inconsistencies.

The use of JSON as an intermediate structured format plays a crucial role in the system architecture. JSON enables efficient data representation by organizing geometric and spatial information into a hierarchical structure. This structured approach allows different modules of the system—such as feature extraction, spatial analysis, object placement, and rendering—to communicate seamlessly. It also ensures modularity, making the system easier to maintain, extend, and integrate with other technologies.

The system demonstrates strong capability in accurately identifying and extracting structural elements from both CAD files and image-based inputs. Using geometric analysis and processing techniques, it can detect key components such as walls, corners, room boundaries, and spatial relationships. The extracted information is then converted into structured JSON data, which serves as a reliable and consistent foundation for 3D model generation.

Another critical aspect of the system is the transformation of 2D coordinates into a 3D spatial environment. This is achieved through coordinate mapping and scaling techniques that ensure proportional accuracy and realistic representation. The system maintains geometric consistency, preserving spatial relationships between different elements, which is essential for generating visually accurate and functionally correct models.

The interior design automation module further enhances the system's capabilities by introducing intelligent placement of furniture and design elements. Based on predefined rules, spatial constraints, and layout logic, the system automatically arranges objects within the generated environment. Advanced features such as collision detection and layout optimization ensure that objects do not overlap and are placed in practical and visually appealing configurations. This not only improves the aesthetic quality of the design but also ensures usability and functional correctness.

The integration of interactive visualization and augmented reality (AR) significantly enhances user experience. Users can explore the generated 3D environments in real time, interact with objects, and analyze spatial arrangements from different perspectives. AR capabilities allow users to visualize designs in real-world contexts, improving understanding and decision-making. This immersive experience is particularly valuable in applications such as client presentations, real estate visualization, and design validation.

From a system design perspective, the framework is modular, scalable, and adaptable. Each component of the system operates independently while maintaining seamless integration with other modules. This modular architecture allows easy updates, addition of new features, and integration with emerging technologies such as AI, cloud computing, and advanced rendering engines.

The system also demonstrates flexibility in handling different types of input data, including CAD files and images, making it versatile for various use cases. Its applications extend beyond interior design to areas such as real estate marketing, construction planning, smart home design, and virtual walkthrough systems.

In conclusion, the proposed system successfully demonstrates the potential of integrating CAD processing, structured data representation, and 3D visualization into a single automated framework. It overcomes the limitations of traditional manual workflows and provides a powerful, efficient, and user-friendly solution for modern design applications. The system not only improves productivity but also enhances creativity, accuracy, and user engagement, making it a valuable tool in the evolving digital design ecosystem.

## 5.2 FUTURE WORK

Although the proposed system achieves significant functionality, efficiency, and practical relevance, there are several opportunities for further enhancement and expansion to improve its capabilities and extend its applicability to more complex and real-world scenarios.

One major area for future improvement is the integration of advanced computer vision and deep learning techniques. Current methods for feature extraction rely primarily on geometric processing, which may struggle with complex or irregular designs. Incorporating deep learning models such as Convolutional Neural Networks (CNNs) and transformer-based architectures can significantly improve the system's ability to detect and interpret complex architectural features. These models can handle variations in design, recognize semantic elements such as doors and windows, and improve robustness in processing noisy or incomplete data.

Another important direction is the development of intelligent AI-based recommendation systems. By analyzing user preferences, historical design data, and current design trends, the system can provide personalized recommendations for furniture layouts, color schemes, lighting arrangements, and decor styles. This would transform the system into a smart design assistant capable of generating optimized and customized solutions for different users.

The integration of real-time data and e-commerce platforms represents another valuable enhancement. By connecting the system to online databases of furniture, materials, and design products, users can access real-time pricing, availability, and product specifications. This would enable features such as cost estimation, product comparison, and direct purchasing, making the system more practical and commercially viable.

Performance optimization is also a critical area for future development. Processing large and complex CAD files can be computationally intensive, especially when combined with real-time rendering and interaction. Techniques such as parallel processing, GPU acceleration, and optimized data structures can significantly improve processing speed and system performance. This is particularly important for professional applications and real-time environments.

Further advancements in augmented reality (AR) and virtual reality (VR) integration can enhance the immersive experience provided by the system. Improvements in environment mapping, object interaction, and spatial tracking can create more realistic and interactive experiences. Features such as gesture control, voice commands, and real-time object manipulation can make the system more intuitive and user-friendly.

The system can also be extended into a cloud-based collaborative platform. This would enable multiple users, such as designers, architects, and clients, to work on the same project simultaneously. Features such as real-time collaboration, design sharing, and remote access would make the system suitable for large-scale and professional projects.

Another promising direction is the incorporation of sustainability and energy-efficiency analysis. The system can provide recommendations for eco-friendly materials, optimal lighting configurations, and energy-efficient layouts. This would support green building practices and align with modern sustainability goals.

Additionally, future work can focus on enhancing user customization and control. While the current system emphasizes automation, allowing users to manually edit layouts, adjust object placements, and fine-tune design elements would provide greater flexibility. This would make the system suitable for both beginners and professional designers with varying levels of expertise.

Finally, expanding the system to support multi-level structures, complex architectural designs, and integration with Building Information Modeling (BIM) systems can further increase its applicability in real-world construction and design workflows.

## REFERENCES


**[1]** Lee, Jongseo, Yudoo Kim, and Il-Young Moon. "Standardization of CAD drawing formats and GeoJSON-based processing for 3D spatial data extraction of underground utilities." *Buildings* 14.12 (2024): 3980.

**[2]** Papagiannopoulos, Ioannis. *JSON application programming interface for discrete event simulation data exchange*. Diss. 2015.

**[3]** Zhou, Shengdi, Tianyi Tang, and Bin Zhou. "CADParser: A Learning Approach of Sequence Modeling for B-Rep CAD." *IJCAI*. 2023.

**[4]** Berger, Elias, et al. "Tiny-CAD-Coder: Leveraging Pre-trained Code Models for Improved CAD Generation."

**[5]** Aubry, Mathieu, et al. "Seeing 3d chairs: exemplar part-based 2d-3d alignment using a large dataset of cad models." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2014.

**[6]** Qin, Fei-wei, et al. "A deep learning approach to the classification of 3D CAD models." *Journal of Zhejiang University-SCIENCE C (Computers & Electronics)* 15.2 (2014): 91-106.

**[7]** Raj, Nikhil, Hema P. Menon, and Abhinand Satheesh. ”Architectural Visualization using Augmented Reality.” 2025 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE). IEEE, 2025.

**[8]** Sharika, T. R., et al. ”Interactive Interior Design Using Augmented Reality and 3D Modeling.” 2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI). IEEE, 2024.

**[9]** Vaidya, Gendlal M., et al. ”Visualization of furniture model using augmented reality.” 2022 Fifth International Conference on Computational Intelligence and Communication Technologies (CCICT). IEEE, 2022.

**[10]** V. A. A. Lesmana, A. Tina, and S. Retno Yanti, “Optimizing AI’s Role in Advancing Interior Design Industry,” Journal of Artificial Intelligence in Architecture, vol. 3, no. 2, pp. 61–71, Aug. 2024.

**[11]** Z. Chen and X. Wang, “Application of AI technology in interior design,” E3S Web of Conferences, vol. 179, p. 02105, 2020.

**[12]** M. Gong, “Application and Practice of Artificial Intelligence Technology in Interior Design,” Applied mathematics and nonlinear sciences, vol. 8, no. 1, pp. 3077–3094, Jan. 2023.

**[13]** Samar Noaman and G. HUSSEIN, “Improving Design Efficiency Using Artificial Intelli gence: A Study on the Role of Artificial Intelligence in Streamlining the Interior Design Process,” International Design Journal (Print), vol. 13, no. 5, pp. 101–114, Sep. 2023.

**[14]** Harihara, Sreenath Panicker, et al. ”PropARt: Augmented Reality-Enabled Multi furniture Placement Application.” International Conference on Computing, Commu nication, Security and Intelligent Systems. Singapore: Springer Nature Singapore, 2024.

# APPENDIX

**Some screenshots of the Code Snippets:**

```
from tkinter import filedialog

def upload_file():
    file_path = filedialog.askopenfilename(
        filetypes=[("All Supported", "*.jpg *.png *.dxf *.pdf")]
    )
    return file_path

file = upload_file()
print("Selected File:", file)
```

**Fig. 1 Code Snippet of File Upload Handling**

```
class Blueprint3DConverter:
    def __init__(self):
        self.corners = {}
        self.walls = []
        self.items = []
        self.scale_factor = 0.01
        self.simplify_tolerance = 2.0
```

**Fig. 2 Code Snippet of Converter Class Initialization**

```python
from PIL import Image

def process_image(self, image_path):
    img = Image.open(image_path)
    width, height = img.size

    corners_list = [
        {"x": 50, "y": 50},
        {"x": width - 50, "y": 50},
        {"x": width - 50, "y": height - 50},
        {"x": 50, "y": height - 50}
    ]

    self.create_walls_from_corners(corners_list)
    self.add_sample_items(width, height)

    return self.get_blueprint_json()
```

**Fig. 3 Code Snippet for Image Processing and Conversion**

```python
import uuid

def generate_id():
    return str(uuid.uuid4())

def create_walls_from_corners(self, corners_list):
    self.corners = {}
    self.walls = []

    for corner in corners_list:
        corner_id = generate_id()
        self.corners[corner_id] = {
            "x": corner["x"] * self.scale_factor,
            "y": corner["y"] * self.scale_factor
        }

    corner_ids = list(self.corners.keys())

    for i in range(len(corner_ids)):
        next_index = (i + 1) % len(corner_ids)
        self.walls.append({
            "corner1": corner_ids[i],
            "corner2": corner_ids[next_index]
        })
```

**Fig. 4 Code Snippet for Wall Generation from Corners**

```python
def add_sample_items(self, width, height):
    scale = self.scale_factor

    self.items.append({
        "item_name": "Sofa",
        "xpos": width * 0.3 * scale,
        "zpos": height * 0.3 * scale
    })

    self.items.append({
        "item_name": "Table",
        "xpos": width * 0.5 * scale,
        "zpos": height * 0.5 * scale
    })
```

**Fig. 5 Code Snippet for Interior Object Placement**

```python
def get_blueprint_json(self):
    return {
        "floorplan": {
            "corners": self.corners,
            "walls": self.walls
        },
        "items": self.items
    }
```

**Fig. 6 Code Snippet for Blueprint JSON Generation**

```python
import json

def save_json(data, filename="blueprint3d.json"):
    with open(filename, "w") as file:
        json.dump(data, file, indent=4)

# Example usage
converter = Blueprint3DConverter()
result = converter.get_blueprint_json()
save_json(result)
```

**Fig. 7 Code Snippet for JSON File Saving**

```python
if __name__ == "__main__":
    converter = Blueprint3DConverter()
    file_path = "sample.png"  # Replace with actual file

    result = converter.process_image(file_path)
    save_json(result)

    print("Conversion Completed Successfully!")
```

**Fig. 8 Code Snippet for System Execution**

# *% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

**Caution: Review required.**

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

## Frequently Asked Questions

**How should I interpret Turnitin's AI writing percentage and false positives?**

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

**What does 'qualifying text' mean?**

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

# 10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Match Groups

- **79** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **11** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

## Top Sources

8% Internet sources

6% Publications

0% Submitted works (Student Papers)

## Integrity Flags

**0 Integrity Flags for Review**

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.